Semantics: Operational, Denotational & Axiomatic

CG

November 7, 2026

Contents

1 Operational semantics **2**
1.1 The behavior of programs . . . 2
1.2 The language SIMP . . . 2
1.3 An abstract machine . . . 4
1.4 Small-step semantics . . . 7
1.5 Big-step semantics . . . 10
1.6 Blocks and local variables . . . 11
1.7 More constructs . . . 12
1.8 Exercises . . . 13

2 Denotational semantics **14**
2.1 Booleans . . . 15
2.2 Numbers and expressions . . . 15
2.3 Other domains . . . 16
2.4 Programs . . . 18
2.5 Commands and fixed points . . . 20
2.6 Exercises . . . 21

3 Axiomatic semantics **21**
3.1 Hoare triples . . . 21
3.2 Rules . . . 22
3.3 Example: the factorial . . . 22
3.4 Exercises . . . 23

4 Control flow **23**
4.1 Expression evaluation . . . 23
4.2 Assignment . . . 24
4.3 Evaluation order . . . 25

4.4 Structured control flow . . . 26
4.5 Sequencing . . . 27
4.6 Selection . . . 28
4.7 Iteration . . . 29
4.8 Recursion . . . 30
4.9 Nondeterminism . . . 30
4.10 Exercises . . . 31

5 Environments, scoping and parameter passing 32
5.1 Variables and bindings . . . 32
5.2 Scopes . . . 32
5.3 Static scoping . . . 33
5.4 Dynamic scoping . . . 35
5.5 Qualified names and access modifiers . . . 36
5.6 Environments and stores . . . 36
5.7 Parameter passing . . . 37
5.8 A small-step rule for calls . . . 38
5.9 Exercises . . . 39

1 Operational semantics

1.1 The behavior of programs

The lecture starts with a line of C++:

```
foo(i++, --i);
```

`i++` returns the value of `i` and then increments it, and `-i` decrements `i` and returns the result. With `i = 0`, what are the arguments of `foo`? If the arguments are evaluated from left to right, `i++` gives 0 and leaves `i = 1`, then `-i` gives 0: the call is `foo(0, 0)`. From right to left, `-i` gives -1, then `i++` gives -1: `foo(-1, -1)`. C++ does not fix the order, and before C++17 the line was undefined. The syntax of the line is clear; its meaning is not. Semantics is the part of the definition of a language that gives meaning to programs.

Semantics come in three flavors: axiomatic semantics, operational semantics (small step and big step, also called natural semantics) and denotational semantics. Besides defining the language, they are used to prove properties of programs, to simulate and explore their behavior and to generate interpreters. This section follows Fernández [3, Ch. 2, 4, 6, 8].

1.2 The language SIMP

SIMP has commands (C), integer expressions (E) and boolean expressions (B):

```
P   ::= C | E | B
```

```
C   ::= l := E | C ; C | if B then C else C | while B do C | skip
E   ::= !l | n | E Op E
Op  ::= + | - | * | /
B   ::= true | false | E Bop E | ¬B | B ∧ B
Bop ::= > | < | =
```

l is a location of the memory (a variable), and $!l$ reads its value. Figure 1 shows the ASTs of two programs of the lecture: the swap `z := !x ; (x := !y ; y := !z)` and the factorial `while !l > 0 do (fac := !fac * !l ; l := !l - 1)`.

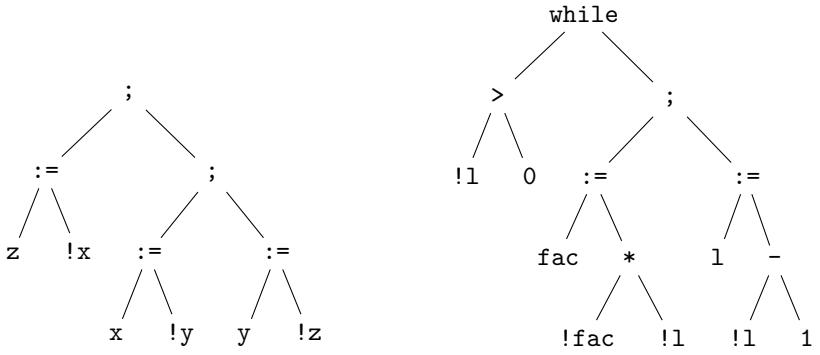


Figure 1: ASTs of the swap (left) and of the factorial (right).

In Rascal the grammar has a priority for the sequence, so that ; binds less than the other commands:

```
layout Layout = [\ \t\n]* !>> [\ \t\n];
keyword Reserved = "if" | "then" | "else" | "while" | "do" | "skip"
                 | "true" | "false" | "repeat" | "times"
                 | "for" | "begin" | "loc" | "end";
lexical Loc = ([a-z][a-z0-9]* !>> [a-z0-9]) \ Reserved;
lexical Num = [0-9]+ !>> [0-9];

start syntax C
  = assign: Loc ":=" E
  | skip: "skip"
  | ifC: "if" B "then" C "else" C
  | whileC: "while" B "do" C
  | repeat: "repeat" E "times" C
  | forC: "for" "(" C ";" B ";" C ")" "{" C "}"
  | block: "begin" "loc" Loc ":=" E ";" C "end"
  | bracket "(" C ")"
  > right seq: C ";" C
  ;

syntax E
  = lit: Num
  | deref: "!" Loc
  | bracket "(" E ")"
  > left (mul: E "*" E | div: E "/" E)
  > left (add: E "+" E | sub: E "-" E)
  ;
```

```
syntax B
  = tt: "true" | ff: "false"
  | gt: E "\>" E | lt: E "\<" E | eq: E "=" E
  | neg: "¬" B
  | bracket "(" B ")"
  > left and: B "∧" B
  ;
```

1.3 An abstract machine

To give meaning to programs, the lecture first describes their behavior with an abstract machine (Figure 2). It has a control stack c with what remains to be done, a results stack r with intermediate values, a processor for arithmetic, and a memory $m : L \to \mathbb{Z}$ from locations to integers.

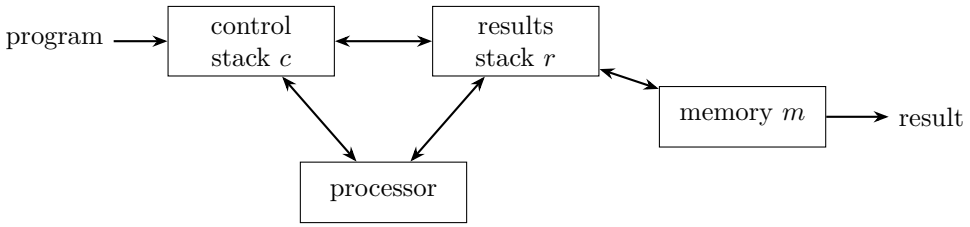


Figure 2: The abstract machine.

Definition 1.1. A **configuration** of the machine is a triple $\langle c, r, m\rangle$, with

$$c ::= \mathit{nil} \mid i \cdot c \qquad i ::= P \mid \mathit{Op} \mid \neg \mid \wedge \mid \mathit{Bop} \mid := \mid \mathit{if} \mid \mathit{while} \qquad r ::= \mathit{nil} \mid P \cdot r \mid l \cdot r.$$

The initial configuration for a command C is $\langle C \cdot \mathit{nil}, \mathit{nil}, m\rangle$. The machine stops at a final configuration $\langle \mathit{nil}, \mathit{nil}, m\rangle$. A computation step transforms a configuration into a new one, $\langle c, r, m\rangle \to \langle c', r', m'\rangle$.

Integer expressions.

$$\begin{array}{lll}
\langle n \cdot c, r, m\rangle & \to \langle c, n \cdot r, m\rangle & \\
\langle !l \cdot c, r, m\rangle & \to \langle c, n \cdot r, m\rangle & \text{if } m(l) = n \\
\langle E_1 \ \mathit{op}\ E_2 \cdot c, r, m\rangle & \to \langle E_1 \cdot E_2 \cdot \mathit{op} \cdot c, r, m\rangle & \\
\langle \mathit{op} \cdot c, n_1 \cdot n_2 \cdot r, m\rangle & \to \langle c, n \cdot r, m\rangle & \text{if } n = n_2 \ \mathit{op}\ n_1
\end{array}$$

E_1 is evaluated first, so its value n_2 ends up below the value n_1 of E_2. The slide writes the condition as $n = n_1 \ \mathit{op}\ n_2$, which gives the right result for $+$ and $*$ but not for $-$, $/$ or the comparisons; its own trace of the factorial computes $4 - 1 = 3$, which is $n_2 - n_1$.

Table 1 is the evaluation of $3 * (4 + 5)$.

Control stack	Results stack	Rule
$3 * (4 + 5) \cdot \mathit{nil}$	nil	split the operation
$3 \cdot (4 + 5) \cdot * \cdot \mathit{nil}$	nil	number
$(4 + 5) \cdot * \cdot \mathit{nil}$	$3 \cdot \mathit{nil}$	split the operation
$4 \cdot 5 \cdot + \cdot * \cdot \mathit{nil}$	$3 \cdot \mathit{nil}$	number
$5 \cdot + \cdot * \cdot \mathit{nil}$	$4 \cdot 3 \cdot \mathit{nil}$	number
$+ \cdot * \cdot \mathit{nil}$	$5 \cdot 4 \cdot 3 \cdot \mathit{nil}$	$4 + 5 = 9$
$* \cdot \mathit{nil}$	$9 \cdot 3 \cdot \mathit{nil}$	$3 * 9 = 27$
nil	$27 \cdot \mathit{nil}$	

Table 1: The machine on $3 * (4 + 5)$.

Boolean expressions.

$$
\begin{array}{lll}
\langle b \cdot c, r, m\rangle & \to \langle c, b \cdot r, m\rangle & \\
\langle \neg B \cdot c, r, m\rangle & \to \langle B \cdot \neg \cdot c, r, m\rangle & \\
\langle \neg \cdot c, b \cdot r, m\rangle & \to \langle c, b' \cdot r, m\rangle & \text{if } b' = \mathit{not}\ b \\
\langle B_1 \wedge B_2 \cdot c, r, m\rangle & \to \langle B_1 \cdot B_2 \cdot \wedge \cdot c, r, m\rangle & \\
\langle \wedge \cdot c, b_1 \cdot b_2 \cdot r, m\rangle & \to \langle c, b \cdot r, m\rangle & \text{if } b = b_2 \wedge b_1 \\
\langle E_1\ \mathit{Bop}\ E_2 \cdot c, r, m\rangle & \to \langle E_1 \cdot E_2 \cdot \mathit{Bop} \cdot c, r, m\rangle & \\
\langle \mathit{Bop} \cdot c, n_1 \cdot n_2 \cdot r, m\rangle & \to \langle c, b \cdot r, m\rangle & \text{if } b = n_2\ \mathit{Bop}\ n_1
\end{array}
$$

(The slide writes the rule for $\neg$ as "b if $b = \mathit{not}\ b$", which never holds; the result is a new value b'.)

Commands. A sequence executes from left to right. An `if` evaluates the condition and keeps both branches on the results stack until it knows which one to run. A `while` evaluates the condition and, if it is true, runs the body and then the whole loop again. An assignment evaluates the expression and then updates the memory:

$$
\begin{array}{ll}
\langle (C_1; C_2) \cdot c, r, m\rangle & \to \langle C_1 \cdot C_2 \cdot c, r, m\rangle \\
\langle (\texttt{if}\ B\ \texttt{then}\ C_1\ \texttt{else}\ C_2) \cdot c, r, m\rangle & \to \langle B \cdot \mathit{if} \cdot c, C_1 \cdot C_2 \cdot r, m\rangle \\
\langle \mathit{if} \cdot c, \mathit{True} \cdot C_1 \cdot C_2 \cdot r, m\rangle & \to \langle C_1 \cdot c, r, m\rangle \\
\langle \mathit{if} \cdot c, \mathit{False} \cdot C_1 \cdot C_2 \cdot r, m\rangle & \to \langle C_2 \cdot c, r, m\rangle \\
\langle (\texttt{while}\ B\ \texttt{do}\ C) \cdot c, r, m\rangle & \to \langle B \cdot \mathit{while} \cdot c, B \cdot C \cdot r, m\rangle \\
\langle \mathit{while} \cdot c, \mathit{True} \cdot B \cdot C \cdot r, m\rangle & \to \langle C \cdot (\texttt{while}\ B\ \texttt{do}\ C) \cdot c, r, m\rangle \\
\langle \mathit{while} \cdot c, \mathit{False} \cdot B \cdot C \cdot r, m\rangle & \to \langle c, r, m\rangle \\
\langle (l := E) \cdot c, r, m\rangle & \to \langle E \cdot := \cdot c, l \cdot r, m\rangle \\
\langle := \cdot c, v \cdot l \cdot r, m\rangle & \to \langle c, r, m[l \mapsto v]\rangle
\end{array}
$$

When the assignment runs, the value of E is on top of l; the slide writes the stack as $l \cdot v \cdot r$, in the opposite order to its own trace.

If $\langle C \cdot \mathit{nil}, \mathit{nil}, m\rangle \to^* \langle \mathit{nil}, \mathit{nil}, m'\rangle$, the program C executed in the memory m terminates successfully and produces m'.

The factorial. With $B = !l > 0$, $C' = \mathit{fac} := !\mathit{fac} * !l; l := !l - 1$ and $m = \{l \mapsto 4, \mathit{fac} \mapsto 1\}$, the first iteration of $C = \texttt{while}\ B\ \texttt{do}\ C'$ is in Table 2. The other three iterations repeat the same pattern, and the machine ends in $\langle \mathit{nil}, \mathit{nil}, m[\mathit{fac} \mapsto 24, l \mapsto 0]\rangle$.

In Rascal each transition is one equation of `machineStep`, with the head of each list as the top of the stack:

Control stack	Results stack	Memory
$C \cdot nil$	nil	m
$!l > 0 \cdot while \cdot nil$	$B \cdot C' \cdot nil$	m
$!l \cdot 0 \cdot > \cdot while \cdot nil$	$B \cdot C' \cdot nil$	m
$0 \cdot > \cdot while \cdot nil$	$4 \cdot B \cdot C' \cdot nil$	m
$> \cdot while \cdot nil$	$0 \cdot 4 \cdot B \cdot C' \cdot nil$	m
$while \cdot nil$	$True \cdot B \cdot C' \cdot nil$	m
$C' \cdot C \cdot nil$	nil	m
$fac :=!fac * !l \cdot l :=!l - 1 \cdot C \cdot nil$	nil	m
$!fac * !l \cdot := \cdot l :=!l - 1 \cdot C \cdot nil$	$fac \cdot nil$	m
$!fac \cdot !l \cdot * \cdot := \cdots$	$fac \cdot nil$	m
$!l \cdot * \cdot := \cdots$	$1 \cdot fac \cdot nil$	m
$* \cdot := \cdots$	$4 \cdot 1 \cdot fac \cdot nil$	m
$:= \cdot l :=!l - 1 \cdot C \cdot nil$	$4 \cdot fac \cdot nil$	m
$l :=!l - 1 \cdot C \cdot nil$	nil	$m[fac \mapsto 4]$
$!l - 1 \cdot := \cdot C \cdot nil$	$l \cdot nil$	$\ldots$
$!l \cdot 1 \cdot - \cdot := \cdot C \cdot nil$	$l \cdot nil$	$\ldots$
$- \cdot := \cdot C \cdot nil$	$1 \cdot 4 \cdot l \cdot nil$	$\ldots$
$:= \cdot C \cdot nil$	$3 \cdot l \cdot nil$	$\ldots$
$C \cdot nil$	nil	$m[fac \mapsto 4, l \mapsto 3]$

Table 2: First iteration of the factorial on the abstract machine.

```
data Item = cI(Cmd c) | eI(Exp e) | bI(BExp b)
          | opI(str op) | notI() | andI() | bopI(str op) | assignI() | ifI() | whileI
    ();
data Val = n(int v) | b(bool truth) | l(str name) | cV(Cmd c) | bV(BExp b);
alias Config = tuple[list[Item] c, list[Val] r, Store m];

bool compare("\>", int x, int y) = x > y;
bool compare("\<", int x, int y) = x < y;
bool compare("=", int x, int y) = x == y;

Config machineStep(<[eI(lit(v)), *c], r, m>)  = <c, [n(v)] + r, m>;
Config machineStep(<[eI(deref(x)), *c], r, m>) = <c, [n(m[x])] + r, m>;
Config machineStep(<[eI(e), *c], r, m>) = <[eI(e1), eI(e2), opI(op)] + c, r, m>
  when <op, e1, e2> := parts(e);
Config machineStep(<[opI(op), *c], [n(n1), n(n2), *r], m>) = <c, [n(apply(op, n2, n1)
    )] + r, m>;

Config machineStep(<[bI(tt()), *c], r, m>) = <c, [b(true)] + r, m>;
Config machineStep(<[bI(ff()), *c], r, m>) = <c, [b(false)] + r, m>;
Config machineStep(<[bI(neg(x)), *c], r, m>) = <[bI(x), notI()] + c, r, m>;
Config machineStep(<[notI(), *c], [b(v), *r], m>) = <c, [b(!v)] + r, m>;
Config machineStep(<[bI(and(x, y)), *c], r, m>) = <[bI(x), bI(y), andI()] + c, r, m>;
Config machineStep(<[andI(), *c], [b(v1), b(v2), *r], m>) = <c, [b(v2 && v1)] + r, m
    >;
Config machineStep(<[bI(gt(e1, e2)), *c], r, m>) = <[eI(e1), eI(e2), bopI("\>")] + c,
     r, m>;
Config machineStep(<[bI(lt(e1, e2)), *c], r, m>) = <[eI(e1), eI(e2), bopI("\<")] + c,
     r, m>;
Config machineStep(<[bI(eq(e1, e2)), *c], r, m>) = <[eI(e1), eI(e2), bopI("=")] + c,
```

```
    r, m>;
Config machineStep(<[bopI(op), *c], [n(n1), n(n2), *r], m>) = <c, [b(compare(op, n2,
    n1))] + r, m>;

Config machineStep(<[cI(skip()), *c], r, m>) = <c, r, m>;
Config machineStep(<[cI(seq(c1, c2)), *c], r, m>) = <[cI(c1), cI(c2)] + c, r, m>;
Config machineStep(<[cI(ifC(x, c1, c2)), *c], r, m>) = <[bI(x), ifI()] + c, [cV(c1),
    cV(c2)] + r, m>;
Config machineStep(<[ifI(), *c], [b(true), cV(c1), cV(_), *r], m>) = <[cI(c1)] + c, r
    , m>;
Config machineStep(<[ifI(), *c], [b(false), cV(_), cV(c2), *r], m>) = <[cI(c2)] + c,
    r, m>;
Config machineStep(<[cI(w:whileC(x, body)), *c], r, m>) = <[bI(x), whileI()] + c, [bV
    (x), cV(body)] + r, m>;
Config machineStep(<[whileI(), *c], [b(true), bV(x), cV(body), *r], m>)
  = <[cI(body), cI(whileC(x, body))] + c, r, m>;
Config machineStep(<[whileI(), *c], [b(false), bV(_), cV(_), *r], m>) = <c, r, m>;
Config machineStep(<[cI(assign(x, e)), *c], r, m>) = <[eI(e), assignI()] + c, [l(x)]
    + r, m>;
Config machineStep(<[assignI(), *c], [n(v), l(x), *r], m>) = <c, r, m + (x: v)>;

tuple[Store, int] runMachine(Cmd p, Store m) {
  Config conf = <[cI(p)], [], m>;
  int steps = 0;
  while (conf.c != []) { conf = machineStep(conf); steps += 1; }
  return <conf.m, steps>;
}

int evalMachine(Exp e) {
  Config conf = <[eI(e)], [], ()>;
  while (conf.c != []) conf = machineStep(conf);
  return conf.r[0].v;
}
```

The tests run $3 * (4 + 5)$, the factorial and $10 - 3$ (which checks the order of the operands).

1.4 Small-step semantics

The machine describes execution with stacks. Small-step semantics describes it directly on the program, as a reduction relation that rewrites a program into a simpler one.

Definition 1.2. A **configuration** of SIMP is a pair $\langle P, s\rangle$ of a program and a store s, a map from locations to values. The program progresses by reductions $\langle P, s\rangle \to \langle P', s'\rangle$ between configurations.

The rules for expressions reduce one operand at a time, from left to right:

$$\frac{s(l) = n}{\langle !l, s\rangle \to \langle n, s\rangle}\ (\textsc{var}) \qquad \frac{n = n_1\ op\ n_2}{\langle n_1\ op\ n_2, s\rangle \to \langle n, s\rangle}\ (\textsc{op}) \qquad \frac{\langle E_1, s\rangle \to \langle E_1', s'\rangle}{\langle E_1\ op\ E_2, s\rangle \to \langle E_1'\ op\ E_2, s'\rangle}\ (\textsc{opL})$$

$$\frac{\langle E_2, s\rangle \to \langle E_2', s'\rangle}{\langle n_1\ op\ E_2, s\rangle \to \langle n_1\ op\ E_2', s'\rangle}\ (\textsc{opR})$$

The rules for booleans are the same with *bop* (BOPL, BOPR), $\wedge$ (ANDL, ANDR) and $\neg$ (NOTARG). Writing n_1 in OPR makes the order fixed: the right operand is reduced only when the left one is already a number.

Commands:

$$\frac{\langle E, s\rangle \to \langle E', s'\rangle}{\langle l := E, s\rangle \to \langle l := E', s'\rangle}\ (:=\mathrm{R}) \qquad \frac{}{\langle l := n, s\rangle \to \langle \texttt{skip}, s[l \mapsto n]\rangle}\ (:=)$$

$$\frac{\langle C_1, s\rangle \to \langle C_1', s'\rangle}{\langle C_1; C_2, s\rangle \to \langle C_1'; C_2, s'\rangle}\ (\textsc{seq}) \qquad \frac{}{\langle \texttt{skip}; C, s\rangle \to \langle C, s\rangle}\ (\textsc{skip})$$

$$\frac{\langle B, s\rangle \to \langle B', s'\rangle}{\langle \texttt{if } B \texttt{ then } C_1 \texttt{ else } C_2, s\rangle \to \langle \texttt{if } B' \texttt{ then } C_1 \texttt{ else } C_2, s'\rangle}\ (\textsc{if})$$

$$\frac{}{\langle \texttt{if } \mathit{True} \texttt{ then } C_1 \texttt{ else } C_2, s\rangle \to \langle C_1, s\rangle}\ (\textsc{if}\mathrm{T}) \qquad \frac{}{\langle \texttt{if } \mathit{False} \texttt{ then } C_1 \texttt{ else } C_2, s\rangle \to \langle C_2, s\rangle}\ (\textsc{if}\mathrm{F})$$

$$\frac{}{\langle \texttt{while } B \texttt{ do } C, s\rangle \to \langle \texttt{if } B \texttt{ then } (C; \texttt{while } B \texttt{ do } C) \texttt{ else skip}, s\rangle}\ (\textsc{while})$$

WHILE has no premise: it unfolds the loop once into an `if`.

Example. The swap with $s(z) = 0$, $s(x) = 1$, $s(y) = 2$:

$$\begin{array}{ll} & \langle z :=!x; (x :=!y; y :=!z), s\rangle \\ \to & \langle z := 1; (x :=!y; y :=!z), s\rangle \\ \to & \langle \texttt{skip}; (x :=!y; y :=!z), s[z \mapsto 1]\rangle \\ \to & \langle x :=!y; y :=!z, s[z \mapsto 1]\rangle \\ \to & \langle x := 2; y :=!z, s[z \mapsto 1]\rangle \\ \to & \langle \texttt{skip}; y :=!z, s[z \mapsto 1, x \mapsto 2]\rangle \\ \to & \langle y :=!z, s[z \mapsto 1, x \mapsto 2]\rangle \\ \to & \langle y := 1, s[z \mapsto 1, x \mapsto 2]\rangle \\ \to & \langle \texttt{skip}, s[z \mapsto 1, x \mapsto 2, y \mapsto 1]\rangle \end{array}$$

At the end $s(z) = 1$, $s(x) = 2$, $s(y) = 1$. (The slides write the store of the middle steps as s; it already contains the updates.)

Every step must be justified by an axiom or a rule. The first step, for example, is the conclusion of the tree in Figure 3: SEQ reduces the first command, :=R reduces its expression, and VAR reads x.

$$\frac{\dfrac{\dfrac{s(x) = 1}{\langle !x, s\rangle \to \langle 1, s\rangle}\ (\textsc{var})}{\langle z :=!x, s\rangle \to \langle z := 1, s\rangle}\ (:=\mathrm{R})}{\langle z :=!x; (x :=!y; y :=!z), s\rangle \to \langle z := 1; (x :=!y; y :=!z), s\rangle}\ (\textsc{seq})$$

Figure 3: The derivation that justifies the first step of the swap.

Final configurations. A configuration with no possible reduction is final: $\langle n, s\rangle$ with n an integer, $\langle b, s\rangle$ with b a boolean, $\langle \texttt{skip}, s\rangle$, and $\langle !l, s\rangle$ with $l \notin \mathrm{dom}(s)$. The last one is a **blocked** configuration. From an initial configuration $\langle P, s\rangle$, the program executes successfully if it reaches $\langle n, s'\rangle$ or $\langle b, s'\rangle$ (for expressions) or $\langle \texttt{skip}, s'\rangle$ (for commands), and it fails if it reaches a blocked configuration.

In Rascal, each `step` function returns the next configuration, or an empty list when the configuration is final:

```
list[tuple[Exp, Store]] stepE(deref(x), Store s) = x in s ? [<lit(s[x]), s>] : [];
list[tuple[Exp, Store]] stepE(lit(_), Store _) = [];
default list[tuple[Exp, Store]] stepE(Exp e, Store s) {
  <op, e1, e2> = parts(e);
  if (lit(n1) := e1, lit(n2) := e2) return [<lit(apply(op, n1, n2)), s>];
  if (lit(_) !:= e1) return [ <rebuild(e, e1p, e2), s1> | <e1p, s1> <- stepE(e1, s)
    ];
  return [ <rebuild(e, e1, e2p), s1> | <e2p, s1> <- stepE(e2, s) ];
}

Exp rebuild(add(_, _), Exp a, Exp b) = add(a, b);
Exp rebuild(sub(_, _), Exp a, Exp b) = sub(a, b);
Exp rebuild(mul(_, _), Exp a, Exp b) = mul(a, b);
Exp rebuild(div(_, _), Exp a, Exp b) = div(a, b);

list[tuple[BExp, Store]] stepB(tt(), Store _) = [];
list[tuple[BExp, Store]] stepB(ff(), Store _) = [];
list[tuple[BExp, Store]] stepB(neg(tt()), Store s) = [<ff(), s>];
list[tuple[BExp, Store]] stepB(neg(ff()), Store s) = [<tt(), s>];
default list[tuple[BExp, Store]] stepB(neg(x), Store s) = [ <neg(y), s1> | <y, s1> <-
     stepB(x, s) ];
list[tuple[BExp, Store]] stepB(and(x, y), Store s) {
  if (x in {tt(), ff()} && y in {tt(), ff()}) return [<x == tt() && y == tt() ? tt()
    : ff(), s>];
  if (x notin {tt(), ff()}) return [ <and(x1, y), s1> | <x1, s1> <- stepB(x, s) ];
  return [ <and(x, y1), s1> | <y1, s1> <- stepB(y, s) ];
}
default list[tuple[BExp, Store]] stepB(BExp x, Store s) {
  <op, e1, e2> = bparts(x);
  if (lit(n1) := e1, lit(n2) := e2) return [<compare(op, n1, n2) ? tt() : ff(), s>];
  if (lit(_) !:= e1) return [ <brebuild(x, e1p, e2), s1> | <e1p, s1> <- stepE(e1, s)
    ];
  return [ <brebuild(x, e1, e2p), s1> | <e2p, s1> <- stepE(e2, s) ];
}

tuple[str, Exp, Exp] bparts(gt(a, b)) = <"\>", a, b>;
tuple[str, Exp, Exp] bparts(lt(a, b)) = <"\<", a, b>;
tuple[str, Exp, Exp] bparts(eq(a, b)) = <"=", a, b>;
BExp brebuild(gt(_, _), Exp a, Exp b) = gt(a, b);
BExp brebuild(lt(_, _), Exp a, Exp b) = lt(a, b);
BExp brebuild(eq(_, _), Exp a, Exp b) = eq(a, b);

list[tuple[Cmd, Store]] stepC(skip(), Store _) = [];
list[tuple[Cmd, Store]] stepC(assign(x, lit(v)), Store s) = [<skip(), s + (x: v)>];
default list[tuple[Cmd, Store]] stepC(assign(x, e), Store s)
```

```
  = [ <assign(x, e1), s1> | <e1, s1> <- stepE(e, s) ];
list[tuple[Cmd, Store]] stepC(seq(skip(), c2), Store s) = [<c2, s>];
default list[tuple[Cmd, Store]] stepC(seq(c1, c2), Store s)
  = [ <seq(c1p, c2), s1> | <c1p, s1> <- stepC(c1, s) ];
list[tuple[Cmd, Store]] stepC(ifC(tt(), c1, _), Store s) = [<c1, s>];
list[tuple[Cmd, Store]] stepC(ifC(ff(), _, c2), Store s) = [<c2, s>];
default list[tuple[Cmd, Store]] stepC(ifC(x, c1, c2), Store s)
  = [ <ifC(y, c1, c2), s1> | <y, s1> <- stepB(x, s) ];
list[tuple[Cmd, Store]] stepC(w:whileC(x, c), Store s)
  = [<ifC(x, seq(c, w), skip()), s>];
```

The tests check that the swap takes the eight steps above and ends with $z = 1$, $x = 2$, $y = 1$, and that `x := !w` stops at a blocked configuration.

1.5 Big-step semantics

Big-step semantics evaluates a program recursively, without showing each step: a transition represents a full evaluation. $\langle P, s\rangle \Downarrow \langle v, s'\rangle$ means that P in the store s evaluates to v and leaves s'.

$$\frac{}{\langle n, s\rangle \Downarrow \langle n, s\rangle}\ (\textsc{num}) \qquad \frac{s(l) = v}{\langle !l, s\rangle \Downarrow \langle v, s\rangle}\ (\textsc{var})$$

$$\frac{\langle E_1, s\rangle \Downarrow \langle n_1, s'\rangle \qquad \langle E_2, s'\rangle \Downarrow \langle n_2, s''\rangle \qquad n = n_1\ op\ n_2}{\langle E_1\ op\ E_2, s\rangle \Downarrow \langle n, s''\rangle}\ (\textsc{op})$$

$$\frac{\langle E, s\rangle \Downarrow \langle n, s'\rangle}{\langle l := E, s\rangle \Downarrow \langle \texttt{skip}, s'[l \mapsto n]\rangle}\ (:=) \qquad \frac{\langle C_1, s\rangle \Downarrow \langle \texttt{skip}, s'\rangle \qquad \langle C_2, s'\rangle \Downarrow \langle \texttt{skip}, s''\rangle}{\langle C_1; C_2, s\rangle \Downarrow \langle \texttt{skip}, s''\rangle}\ (\textsc{seq})$$

$$\frac{\langle B, s\rangle \Downarrow \langle \mathit{True}, s'\rangle \qquad \langle C_1, s'\rangle \Downarrow \langle \texttt{skip}, s''\rangle}{\langle \texttt{if}\ B\ \texttt{then}\ C_1\ \texttt{else}\ C_2, s\rangle \Downarrow \langle \texttt{skip}, s''\rangle}\ (\textsc{if}\mathrm{T})$$

$$\frac{\langle B, s\rangle \Downarrow \langle \mathit{True}, s_1\rangle \qquad \langle C, s_1\rangle \Downarrow \langle \texttt{skip}, s_2\rangle \qquad \langle \texttt{while}\ B\ \texttt{do}\ C, s_2\rangle \Downarrow \langle \texttt{skip}, s_3\rangle}{\langle \texttt{while}\ B\ \texttt{do}\ C, s\rangle \Downarrow \langle \texttt{skip}, s_3\rangle}\ (\textsc{while}\mathrm{T})$$

$$\frac{\langle B, s\rangle \Downarrow \langle \mathit{False}, s'\rangle}{\langle \texttt{while}\ B\ \texttt{do}\ C, s\rangle \Downarrow \langle \texttt{skip}, s'\rangle}\ (\textsc{while}\mathrm{F})$$

IFF is like IFT with *False* and C_2, and the boolean rules are like OP.

Example. The derivation of the swap is built from the leaves up. With $s_1 = s[z \mapsto 1]$ and $s_2 = s[z \mapsto 1, x \mapsto 2]$:

$\langle !x, s\rangle \Downarrow \langle 1, s\rangle$	(var)
$\langle z := !x, s\rangle \Downarrow \langle \texttt{skip}, s_1\rangle$	(:=)
$\langle !y, s_1\rangle \Downarrow \langle 2, s_1\rangle$	(var)
$\langle x := !y, s_1\rangle \Downarrow \langle \texttt{skip}, s_2\rangle$	(:=)
$\langle !z, s_2\rangle \Downarrow \langle 1, s_2\rangle$	(var)
$\langle y := !z, s_2\rangle \Downarrow \langle \texttt{skip}, s_2[y \mapsto 1]\rangle$	(:=)
$\langle z := !x; x := !y; y := !z, s\rangle \Downarrow \langle \texttt{skip}, s[z \mapsto 1, x \mapsto 2, y \mapsto 1]\rangle$	(seq), twice

Figure 4 draws the tree for the first two assignments.

$$\dfrac{\dfrac{\dfrac{}{\langle !x, s\rangle \Downarrow \langle 1, s\rangle}\,(\textsc{var})}{\langle z := !x, s\rangle \Downarrow \langle \texttt{skip}, s_1\rangle}\,(:=) \qquad \dfrac{\dfrac{}{\langle !y, s_1\rangle \Downarrow \langle 2, s_1\rangle}\,(\textsc{var})}{\langle x := !y, s_1\rangle \Downarrow \langle \texttt{skip}, s_2\rangle}\,(:=)}{\langle z := !x; x := !y, s\rangle \Downarrow \langle \texttt{skip}, s_2\rangle}\,(\textsc{seq})$$

Figure 4: Big-step derivation of the first two assignments of the swap.

In Rascal big-step semantics is a recursive evaluator with one equation per rule:

```
int evalE(lit(v), Store _) = v;
int evalE(deref(x), Store s) = s[x];
default int evalE(Exp e, Store s) { <op, e1, e2> = parts(e); return apply(op, evalE(
    e1, s), evalE(e2, s)); }

bool evalB(tt(), Store _) = true;
bool evalB(ff(), Store _) = false;
bool evalB(neg(x), Store s) = !evalB(x, s);
bool evalB(and(x, y), Store s) = evalB(x, s) && evalB(y, s);
default bool evalB(BExp x, Store s) { <op, e1, e2> = bparts(x); return compare(op,
    evalE(e1, s), evalE(e2, s)); }

Store evalC(skip(), Store s) = s;
Store evalC(assign(x, e), Store s) = s + (x: evalE(e, s));
Store evalC(seq(c1, c2), Store s) = evalC(c2, evalC(c1, s));
Store evalC(ifC(x, c1, c2), Store s) = evalB(x, s) ? evalC(c1, s) : evalC(c2, s);
Store evalC(w:whileC(x, c), Store s) = evalB(x, s) ? evalC(w, evalC(c, s)) : s;
```

Small step is the natural model of a debugger: it shows each intermediate state, and it can describe programs that do not terminate (an infinite sequence of steps). Big step is the natural model of an interpreter, and a program that does not terminate simply has no derivation.

1.6 Blocks and local variables

Blocks give locality to programs: a variable declared in a block is only visible and valid inside it. SIMP is extended with blocks and with allocation of new locations:

```
C ::= ... | begin loc l := E ; C end
E ::= ... | ref E
```

$$\dfrac{\langle E, s\rangle \Downarrow \langle v, s'\rangle}{\langle \texttt{ref } E, s\rangle \Downarrow \langle l, s'[l \mapsto v]\rangle}\,(\textsc{ref}) \qquad l \notin \mathrm{dom}(s')$$

$$\dfrac{\langle E, s\rangle \Downarrow \langle v, s'\rangle \qquad \langle C\{z \mapsto l\}, s'[l \mapsto v]\rangle \Downarrow \langle \texttt{skip}, s''[l \mapsto v']\rangle}{\langle \texttt{begin loc } z := E; C \texttt{ end}, s\rangle \Downarrow \langle \texttt{skip}, s''\rangle}\,(\textsc{loc})$$

with $l \notin \mathrm{dom}(s') \cup \mathrm{dom}(s'') \cup \mathit{locations}(C)$. $C\{z \mapsto l\}$ replaces the name z by the new location l in C, and the final store drops l: the local variable does not exist after the block. For the swap with a local variable,

```
begin
  loc z := !x;
  x := !y;
  y := !z
end
```

the body runs in $s'[l \mapsto s(x)]$ and gives $s_1 = s[l \mapsto s(x), x \mapsto s(y), y \mapsto s(x)]$; the whole program gives $s_2 = s[x \mapsto s(y), y \mapsto s(x)]$, with no z.

```
Store evalC(block(z, e, c), Store s) {
  int v = evalE(e, s);
  str fresh = "l<size(s)>";
  while (fresh in s) fresh += "\'";
  Cmd body = visit (c) { case assign(z, ex) => assign(fresh, ex) case deref(z) =>
    deref(fresh) };
  Store s2 = evalC(body, s + (fresh: v));
  return delete(s2, fresh);
}
```

1.7 More constructs

This subsection goes beyond the slides.

repeat. The command `repeat` E `times` C runs C as many times as the value of E, evaluated once. In small step:

$$\frac{\langle E, s\rangle \to \langle E', s'\rangle}{\langle \texttt{repeat } E \texttt{ times } C, s\rangle \to \langle \texttt{repeat } E' \texttt{ times } C, s'\rangle}\ (\textsc{repeat-e})$$

$$\frac{n \le 0}{\langle \texttt{repeat } n \texttt{ times } C, s\rangle \to \langle \texttt{skip}, s\rangle}\ (\textsc{repeat-fin})$$

$$\frac{n > 0 \qquad m = n - 1}{\langle \texttt{repeat } n \texttt{ times } C, s\rangle \to \langle C; \texttt{repeat } m \texttt{ times } C, s\rangle}\ (\textsc{repeat-paso})$$

Once E is a numeral it is never evaluated again, even if C changes the variables of E. The base case is $n \le 0$ and not $n = 0$: with only $n = 0$, a negative count would leave the configuration blocked.

```
list[tuple[Cmd, Store]] stepC(repeat(lit(v), c), Store s)
  = v <= 0 ? [<skip(), s>]
           : [<seq(c, repeat(lit(v - 1), c)), s>];
default list[tuple[Cmd, Store]] stepC(repeat(e, c), Store s)
  = [ <repeat(e1, c), s1> | <e1, s1> <- stepE(e, s) ];
```

```
Store evalC(repeat(e, c), Store s) {
  int v = evalE(e, s);
  return v <= 0 ? s : evalC(repeat(lit(v - 1), c), evalC(c, s));
}
```

The tests run `repeat !k times (x := !x + 1; k := !k + 1)` with $k = 2$, which runs the body twice although k grows, and a negative count.

for. A loop $\texttt{for}\,(C_i; B; C_s)\,\{C_b\}$ is syntactic sugar for $C_i; \texttt{while}\ B\ \texttt{do}\ (C_b; C_s)$. With its own rules, the initialization is reduced first and then the loop unfolds into an `if`:

$$\frac{\langle C_i, s\rangle \to \langle C_i', s'\rangle}{\langle \texttt{for}\,(C_i; B; C_s)\,\{C_b\}, s\rangle \to \langle \texttt{for}\,(C_i'; B; C_s)\,\{C_b\}, s'\rangle}\ \textsc{For-Init}$$

$$\frac{}{\langle \texttt{for}\,(\texttt{skip}; B; C_s)\,\{C_b\}, s\rangle \to \langle \texttt{if}\ B\ \texttt{then}\ (C_b; C_s; \texttt{for}\,(\texttt{skip}; B; C_s)\,\{C_b\})\ \texttt{else skip}, s\rangle}\ \textsc{For-Skip}$$

```
list[tuple[Cmd, Store]] stepC(forC(skip(), x, st, body), Store s)
  = [<ifC(x, seq(body, seq(st, forC(skip(), x, st, body))), skip()), s>];
default list[tuple[Cmd, Store]] stepC(forC(init, x, st, body), Store s)
  = [ <forC(i1, x, st, body), s1> | <i1, s1> <- stepC(init, s) ];
```

Lists. Big step also defines functions on data. With the constructors **nil** and $\textbf{cons}(h, t)$, adding an element at the end of a list and appending two lists are

$$\frac{}{\langle \textbf{concat}(e, \textbf{nil}),\, \sigma\rangle \Downarrow \textbf{cons}(e, \textbf{nil})}\ \textsc{Concat-Nil}$$

$$\frac{\langle \textbf{concat}(e, t),\, \sigma\rangle \Downarrow t'}{\langle \textbf{concat}(e, \textbf{cons}(h, t)),\, \sigma\rangle \Downarrow \textbf{cons}(h, t')}\ \textsc{Concat-Cons} \qquad \frac{}{\langle \textbf{append}(\textbf{nil}, \ell_2),\, \sigma\rangle \Downarrow \ell_2}\ \textsc{Append-Nil}$$

$$\frac{\langle \textbf{append}(t_1, \ell_2),\, \sigma\rangle \Downarrow \ell'}{\langle \textbf{append}(\textbf{cons}(h, t_1), \ell_2),\, \sigma\rangle \Downarrow \textbf{cons}(h, \ell')}\ \textsc{Append-Cons}$$

1.8 Exercises

Exercise 1.3. Draw the AST of `while !l > 0 do (factorial := !factorial * !l; l := !l - 1)`.

Solution. It is the right tree of Figure 1, with `factorial` in place of `fac`.

Exercise 1.4. Define the transitions of the abstract machine for `;`, `if`, `while` and assignment.

Solution. They are the command rules of the abstract machine above. The idea in each one: put the parts that must run first on the control stack, and keep on the results stack what will be needed later (the branches of the `if`, the condition and the body of the `while`, the location of the assignment).

Exercise 1.5. Run `y := !x + 1 ; x := !y * 2` from $\sigma = \{x \mapsto 3, y \mapsto 0\}$ in small step and in big step.

Solution. In small step, with $\sigma_1 = \sigma[y \mapsto 4]$ and $\sigma_2 = \sigma_1[x \mapsto 8]$:

$$\begin{aligned}
& \langle y :=\!!x + 1; x :=\!!y * 2, \sigma\rangle \\
\to\ & \langle y := 3 + 1; x :=\!!y * 2, \sigma\rangle \quad \text{(seq, :=R, opL, var)} \\
\to\ & \langle y := 4; x :=\!!y * 2, \sigma\rangle \quad \text{(seq, :=R, op)} \\
\to\ & \langle \texttt{skip}; x :=\!!y * 2, \sigma_1\rangle \quad \text{(seq, :=)} \\
\to\ & \langle x :=\!!y * 2, \sigma_1\rangle \quad \text{(skip)} \\
\to\ & \langle x := 4 * 2, \sigma_1\rangle \to \langle x := 8, \sigma_1\rangle \to \langle \texttt{skip}, \sigma_2\rangle
\end{aligned}$$

In big step there is one tree: SEQ with two premises, each an assignment whose premise evaluates $!x + 1 \Downarrow 4$ in σ and $!y * 2 \Downarrow 8$ in σ_1. Both give $\{x \mapsto 8, y \mapsto 4\}$.

Exercise 1.6. Give small-step rules for `for` E_1 `to` E_2 `do` C, which evaluates E_1 and E_2 once and runs C for each integer from n_1 to n_2.

Solution. Reduce the bounds first, then unfold:

$$\frac{\langle E_1, s\rangle \to \langle E_1', s'\rangle}{\langle \texttt{for } E_1 \texttt{ to } E_2 \texttt{ do } C, s\rangle \to \langle \texttt{for } E_1' \texttt{ to } E_2 \texttt{ do } C, s'\rangle}$$

$$\frac{\langle E_2, s\rangle \to \langle E_2', s'\rangle}{\langle \texttt{for } n_1 \texttt{ to } E_2 \texttt{ do } C, s\rangle \to \langle \texttt{for } n_1 \texttt{ to } E_2' \texttt{ do } C, s'\rangle} \qquad \frac{n_1 > n_2}{\langle \texttt{for } n_1 \texttt{ to } n_2 \texttt{ do } C, s\rangle \to \langle \texttt{skip}, s\rangle}$$

$$\frac{n_1 \le n_2}{\langle \texttt{for } n_1 \texttt{ to } n_2 \texttt{ do } C, s\rangle \to \langle C; \texttt{for } (n_1 + 1) \texttt{ to } n_2 \texttt{ do } C, s\rangle}$$

Unfolding before the bounds are numbers would evaluate them again in each iteration. Without the rule for $n_1 > n_2$ an empty range would be blocked.

Exercise 1.7. Write big-step rules for $\mathbf{reverse}(\ell)$, using `concat`, and prove

$$\langle \mathbf{concat}(3, \mathbf{cons}(1, \mathbf{cons}(2, \mathbf{nil}))), \sigma\rangle \Downarrow \mathbf{cons}(1, \mathbf{cons}(2, \mathbf{cons}(3, \mathbf{nil}))).$$

Solution.

$$\frac{}{\mathbf{reverse}(\mathbf{nil}) \Downarrow \mathbf{nil}}\ \textsc{Rev-Nil} \qquad \frac{\mathbf{reverse}(t) \Downarrow t' \qquad \mathbf{concat}(h, t') \Downarrow \ell'}{\mathbf{reverse}(\mathbf{cons}(h, t)) \Downarrow \ell'}\ \textsc{Rev-Cons}$$

For the proof, CONCAT-CONS twice and CONCAT-NIL once: $\mathbf{concat}(3, \mathbf{nil}) \Downarrow \mathbf{cons}(3, \mathbf{nil})$, then $\mathbf{concat}(3, \mathbf{cons}(2, \mathbf{nil})) \Downarrow \mathbf{cons}(2, \mathbf{cons}(3, \mathbf{nil}))$, then the goal.

2 Denotational semantics

Syntax describes the form of the strings of a language, and semantics gives meaning to its elements, usually following the grammar. Operational semantics describes the behavior of each element on a machine, through actions guided by the derivation. Denotational semantics instead defines a map from symbols and grammar rules to mathematical domains, and computes the value of an entity from its derivation.

Definition 2.1. Given a domain D, the **denotational semantics** of a language is a partial function $m : \Sigma^* \to D$ that gives each word of the language a value in D. The function is **compositional**: the meaning of a construct is computed from the meanings of its parts.

For an imperative language the domain of a program is a function on memories: $m(Exp)$: $Environment \times Store \to Store$. The examples below are written as Scheme functions over abstract syntax trees written as S-expressions; each function is the semantic function m of its language.

2.1 Booleans

The language of booleans has expressions such as $(x \;\&\&\; true) \;||\; (not\ y \;\&\&\; false)$. The lecture contrasts it with the Haskell type `data Bool = True | False`, whose operations are defined by cases:

$$\begin{array}{ll} False \;\&\&\; x = False & True \;\&\&\; x = x \\ False \;||\; x = x & True \;||\; x = True \\ not\ False = True & not\ True = False \end{array}$$

The expressions are syntax; `Bool` and its operations are the domain. The semantic function maps each expression to an element of the domain, given the values of the variables:

```
(define (m-bool b env)
  (cond ((eq? b 'true) #t)
        ((eq? b 'false) #f)
        ((symbol? b) (cdr (assq b env)))
        ((eq? (car b) 'and) (and (m-bool (cadr b) env) (m-bool (caddr b) env)))
        ((eq? (car b) 'or)  (or  (m-bool (cadr b) env) (m-bool (caddr b) env)))
        ((eq? (car b) 'not) (not (m-bool (cadr b) env)))))
```

2.2 Numbers and expressions

For $E ::= E + E \mid (0..9)^+$ the meaning is a natural number, $m : E \to \mathbb{N}$:

$$m[\![0]\!] = 0,\ \ldots,\ m[\![9]\!] = 9 \qquad m[\![N\,D]\!] = m[\![N]\!] * 10 + m[\![D]\!] \qquad m[\![E_1 + E_2]\!] = m[\![E_1]\!] +_{\mathbb{N}} m[\![E_2]\!]$$

The $+$ inside the brackets is a symbol of the language; $+_{\mathbb{N}}$ is the addition of natural numbers. The rule for $N\,D$ reads a number digit by digit: $m[\![123]\!] = (1 * 10 + 2) * 10 + 3$.

```
(define (m-digit d) (- (char->integer d) (char->integer #\0)))

(define (m-num digits)
  (let loop ((ds (string->list digits)) (acc 0))
    (if (null? ds) acc (loop (cdr ds) (+ (* acc 10) (m-digit (car ds)))))))

(define (m-e e)
  (if (string? e)
      (m-num e)
      (+ (m-e (cadr e)) (m-e (caddr e)))))
```

Rule	Equation	Explanation
$E ::= E + T$	$m[\![E + T]\!] = m[\![E]\!] + m[\![T]\!]$	sum of the subexpression and the term
$E ::= T$	$m[\![E]\!] = m[\![T]\!]$	the expression is its term
$T ::= T * F$	$m[\![T * F]\!] = m[\![T]\!] \times m[\![F]\!]$	product of the term and the factor
$T ::= F$	$m[\![T]\!] = m[\![F]\!]$	the term is its factor
$F ::= num$	$m[\![num]\!] = num.val$	the value of the number
$F ::= (E)$	$m[\![(E)]\!] = m[\![E]\!]$	parentheses keep the value

Table 3: Denotational equations for infix expressions.

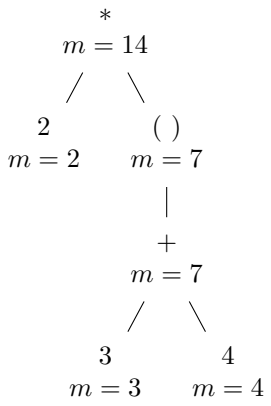


Figure 5: The meaning of each node of $2 * (3 + 4)$.

Infix expressions. For the grammar $E ::= E + T \mid T$, $T ::= T * F \mid F$, $F ::= num \mid (E)$, where only *num* has a value (*num.val*), there is one equation per rule (Table 3). Figure 5 shows the meaning of each node of $2 * (3 + 4)$.

```
(define (m-infix e)
  (cond ((number? e) e)
        ((eq? (car e) 'paren) (m-infix (cadr e)))
        ((eq? (car e) '+) (+ (m-infix (cadr e)) (m-infix (caddr e))))
        ((eq? (car e) '*) (* (m-infix (cadr e)) (m-infix (caddr e))))))
```

2.3 Other domains

The domain does not have to be numbers. The lecture gives several examples where the meaning of an expression is something else.

Expression trees. With variables a to z, the meaning of an expression can be its tree, built with *leafInt*(n), *leafVar*(c) and *tree*(op, t_1, t_2): $m[\![E + T]\!] = tree(\text{`}+', m[\![E]\!], m[\![T]\!])$, $m[\![NUM]\!] = leafInt(NUM.val)$ and $m[\![VAR]\!] = leafVar(VAR.val)$. This is the AST construction of the previous block, written as a semantic function.

```
(define (leaf-int n) (list 'int n))
(define (leaf-var c) (list 'var c))
(define (tree op t1 t2) (list op t1 t2))
```

```
(define (m-tree e)
  (cond ((number? e) (leaf-int e))
        ((symbol? e) (leaf-var e))
        (else (tree (car e) (m-tree (cadr e)) (m-tree (caddr e))))))

(define (eval-tree t x)
  (case (car t)
    ((int) (cadr t))
    ((var) x)
    ((+) (+ (eval-tree (cadr t) x) (eval-tree (caddr t) x)))
    ((*) (* (eval-tree (cadr t) x) (eval-tree (caddr t) x)))
    ((^) (expt (eval-tree (cadr t) x) (eval-tree (caddr t) x)))))
```

Derivatives with pairs. The meaning of an expression can be a pair $\langle a, b\rangle$ where a is its tree and b the tree of its derivative with respect to x:

$$\begin{array}{ll}
\partial[\![E+T]\!] = \langle \mathit{fst}(E)+\mathit{fst}(T),\ \mathit{snd}(E)+\mathit{snd}(T)\rangle & \text{sum rule}\\
\partial[\![T*F]\!] = \langle \mathit{fst}(T)*\mathit{fst}(F),\ \mathit{snd}(T)*\mathit{fst}(F)+\mathit{fst}(T)*\mathit{snd}(F)\rangle & \text{product rule}\\
\partial[\![P^n]\!] = \langle \mathit{fst}(P)^n,\ n*\mathit{fst}(P)^{n-1}*\mathit{snd}(P)\rangle & \text{power rule}
\end{array}$$

where E, T, F and P in the right side stand for their meanings. The slide writes the product rule with $\mathit{snd}(E)$ and $\mathit{fst}(E)$ in the second component; the operands are T and F. It also writes $n*P^{n-1}$, where the base is the tree $\mathit{fst}(P)$.

```
(define (d e)
  (cond ((number? e) (cons (leaf-int e) (leaf-int 0)))
        ((eq? e 'x)  (cons (leaf-var 'x) (leaf-int 1)))
        ((symbol? e) (cons (leaf-var e) (leaf-int 0)))
        ((eq? (car e) '+)
         (let ((a (d (cadr e))) (b (d (caddr e))))
           (cons (tree '+ (car a) (car b)) (tree '+ (cdr a) (cdr b)))))
        ((eq? (car e) '*)
         (let ((a (d (cadr e))) (b (d (caddr e))))
           (cons (tree '* (car a) (car b))
                 (tree '+ (tree '* (cdr a) (car b)) (tree '* (car a) (cdr b))))))
        ((eq? (car e) '^)
         (let ((a (d (cadr e))) (n (caddr e)))
           (cons (tree '^ (car a) (leaf-int n))
                 (tree '* (tree '* (leaf-int n) (tree '^ (car a) (leaf-int (- n 1))))
                       (cdr a)))))))
```

The tests evaluate both trees of $x*x+3*x$ at $x=2$ and get $f(2)=10$ and $f'(2)=7$.

Complex numbers. A complex number has a rectangular form (real, imaginary) and a polar form (angle, magnitude). The meaning is a triple (T, V_1, V_2) with $T=1$ for rectangular and $T=0$ for polar. Addition adds the parts in rectangular form, $E[\![E_1+E_2]\!] = (1, \mathit{real}(m[\![E_1]\!]) + \mathit{real}(m[\![E_2]\!]), \mathit{img}(m[\![E_1]\!]) + \mathit{img}(m[\![E_2]\!]))$, and multiplication adds angles and multiplies magnitudes in polar form. The functions *real*, *img*, *ang* and *mag* convert when the argument is in the other form.

```
(define (rect r i) (list 1 r i))
```

```
(define (polar a m) (list 0 a m))
(define (real z) (if (= (car z) 1) (cadr z) (* (caddr z) (cos (cadr z)))))
(define (img z)  (if (= (car z) 1) (caddr z) (* (caddr z) (sin (cadr z)))))
(define (ang z)  (if (= (car z) 0) (cadr z) (atan (caddr z) (cadr z))))
(define (mag z)  (if (= (car z) 0) (caddr z) (sqrt (+ (* (cadr z) (cadr z)) (* (caddr
     z) (caddr z))))))

(define (m-complex e)
  (cond ((eq? (car e) 'rect) (rect (cadr e) (caddr e)))
        ((eq? (car e) 'polar) (polar (cadr e) (caddr e)))
        ((eq? (car e) '+)
         (let ((z1 (m-complex (cadr e))) (z2 (m-complex (caddr e))))
           (rect (+ (real z1) (real z2)) (+ (img z1) (img z2)))))
        ((eq? (car e) '*)
         (let ((z1 (m-complex (cadr e))) (z2 (m-complex (caddr e))))
           (polar (+ (ang z1) (ang z2)) (* (mag z1) (mag z2)))))))
```

S-expressions. The S-expressions of Lisp, restricted to $S ::= (+\ L) \mid (*\ L) \mid num$, $L ::= S \mid L\,S$, have two domains: a number for S and a list of numbers for L:

$$m[\![S \to num]\!] = num.val \qquad m[\![S \to (+\ L)]\!] = apply(+, m[\![L]\!])$$
$$m[\![S \to (*\ L)]\!] = apply(*, m[\![L]\!]) \qquad m[\![L \to S]\!] = insertEnd((), m[\![S]\!])$$
$$m[\![L \to L\,S]\!] = insertEnd(m[\![L]\!], m[\![S]\!])$$

The slide repeats $S \to (+\ L)$ in the rule for $*$.

```
(define (m-s s)
  (cond ((number? s) s)
        ((eq? (car s) '+) (apply + (m-l (cdr s))))
        ((eq? (car s) '*) (apply * (m-l (cdr s))))))

(define (m-l l)
  (if (null? l) '() (cons (m-s (car l)) (m-l (cdr l)))))
```

Sets of digits. For $S ::= \{\} \mid \{Nums\}$, $Nums ::= num \mid Nums, num$, the meaning of a set is a 10-tuple of booleans whose i-th element says whether i is in the set. The empty set is a new tuple of *false*, and each number sets its position. Union, intersection and complement work position by position: $m[\![S_1 \cup S_2]\!]_i = m[\![S_1]\!]_i \,||\, m[\![S_2]\!]_i$, $m[\![S_1 \cap S_2]\!]_i = m[\![S_1]\!]_i \,\&\&\, m[\![S_2]\!]_i$, $m[\![S^c]\!]_i = \neg m[\![S]\!]_i$.

```
(define (new-tuple) (make-vector 10 #f))
(define (m-set nums)
  (let ((t (new-tuple)))
    (for-each (lambda (n) (vector-set! t n #t)) nums)
    t))
(define (set-union t1 t2) (vector-map (lambda (a b) (or a b)) t1 t2))
(define (set-inter t1 t2) (vector-map (lambda (a b) (and a b)) t1 t2))
(define (set-compl t) (vector-map not t))
```

2.4 Programs

Assignment sequences as memories. For

```
P  ::= begin As end
As ::= As ; A | A
A  ::= var := E
E  ::= E + T | ...
```

the meaning of a program is the memory after executing it. [ ] is the empty memory, $Mem[v/var]$ the memory after assigning v to var, and $Mem[var]$ the value of var (zero if undefined):

$$m[\![\texttt{begin } As \texttt{ end}]\!] = assign[\![As]\!]\,[\,]$$
$$assign[\![As; A]\!]\,Mem = m[\![A]\!]\,(assign[\![As]\!]\,Mem)$$
$$m[\![var := E]\!]\,Mem = Mem[m[\![E]\!]Mem/var.val]$$

Each assignment is a function from memories to memories, and a sequence is their composition.

```
(define (lookup mem var) (let ((p (assq var mem))) (if p (cdr p) 0)))

(define (m-exp e mem)
  (cond ((number? e) e)
        ((symbol? e) (lookup mem e))
        ((eq? (car e) '+) (+ (m-exp (cadr e) mem) (m-exp (caddr e) mem)))
        ((eq? (car e) '*) (* (m-exp (cadr e) mem) (m-exp (caddr e) mem)))))

(define (assign as mem)
  (if (null? as)
      mem
      (let* ((a (car as)) (var (cadr a)) (e (caddr a)))
        (assign (cdr as) (cons (cons var (m-exp e mem)) mem)))))

(define (m-program p) (assign (cdr p) '()))
```

Translation as meaning. With the same grammar, the meaning of a program can be its translation to code for a stack machine, with the instructions *ASG var* (pop and assign), *GET var* and *PUSH num* (push), and *PLUS* and *TIMES* (pop two values, push the result). Code is built by appending instructions with *code.add*:

$$m[\![\texttt{begin } As \texttt{ end}]\!] = assign[\![As]\!]\,\langle\rangle$$
$$assign[\![As; A]\!]\,CODE = m[\![A]\!]\,(assign[\![As]\!]\,CODE)$$
$$m[\![var := E]\!]\,CODE = (m[\![E]\!]\,CODE).add(ASG, var.val)$$
$$m[\![E + T]\!]\,CODE = (m[\![T]\!]\,(m[\![E]\!]\,CODE)).add(PLUS)$$
$$m[\![T * F]\!]\,CODE = (m[\![F]\!]\,(m[\![T]\!]\,CODE)).add(TIMES)$$
$$m[\![var]\!]\,CODE = CODE.add(GET, var.val) \qquad m[\![num]\!]\,CODE = CODE.add(PUSH, num.val)$$

The slide writes the rule for $T * F$ with $m[\![E]\!]$ and $m[\![T]\!]$, copied from the rule for $E + T$, and passes $CODE_2$ to the function that produces $CODE_2$; the code of the left operand has to be the input of the code of the right one. Figure 6 shows the code for `y := x + 4` and the stack while it runs.

```
(define (emit code . instruction)
  (append code (list instruction)))

(define (m-code-exp e code)
  (cond ((number? e) (emit code 'push e))
```

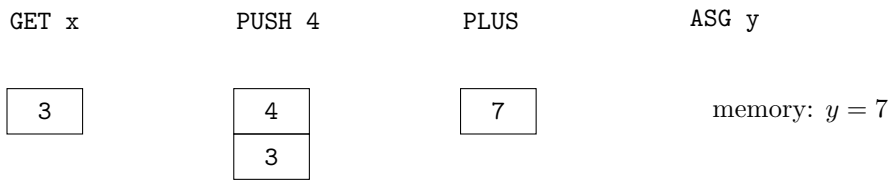


Figure 6: The code of `y := x + 4` with $x = 3$, and the stack after each instruction (top first).

```
        ((symbol? e) (emit code 'get e))
        ((eq? (car e) '+)
         (emit (m-code-exp (caddr e) (m-code-exp (cadr e) code)) 'plus))
        ((eq? (car e) '*)
         (emit (m-code-exp (caddr e) (m-code-exp (cadr e) code)) 'times))))

(define (m-code p)
  (let loop ((as (cdr p)) (code '()))
    (if (null? as)
        code
        (let ((a (car as)))
          (loop (cdr as)
                (emit (m-code-exp (caddr a) code) 'asg (cadr a)))))))

(define (run code)
  (let loop ((code code) (stack '()) (mem '()))
    (if (null? code)
        mem
        (let* ((i (car code)) (rest (cdr code)))
          (case (car i)
            ((push) (loop rest (cons (cadr i) stack) mem))
            ((get) (loop rest (cons (lookup mem (cadr i)) stack) mem))
            ((plus) (loop rest (cons (+ (car stack) (cadr stack)) (cddr stack)) mem))
            ((times) (loop rest (cons (* (car stack) (cadr stack)) (cddr stack)) mem)
    )
            ((asg) (loop rest (cdr stack)
                         (cons (cons (cadr i) (car stack)) mem)))))))))
```

The test runs the code of a program on the machine and checks that it gives the same memory as the direct semantics.

2.5 Commands and fixed points

This subsection goes beyond the slides. For SIMP the domain of a command is a function from states to states, $\Sigma = \mathit{Var} \to \mathit{Val}$:

$$[\![C]\!] : \Sigma \to \Sigma_\perp,$$

where $\Sigma_\perp = \Sigma \cup \{\perp\}$ adds $\perp$ for programs that do not terminate. Assignment, sequence and conditional are direct: $[\![l := E]\!]\sigma = \sigma[l \mapsto [\![E]\!]\sigma]$, $[\![C_1; C_2]\!] = [\![C_2]\!] \circ [\![C_1]\!]$. The loop `while` b `do` C is the least fixed point of

$$\Phi(f)(\sigma) = \begin{cases} f([\![C]\!]\sigma) & \text{if } [\![b]\!]\sigma = \mathit{true} \\ \sigma & \text{if } [\![b]\!]\sigma = \mathit{false} \end{cases}$$

The approximations $\Phi^0 = \bot$, $\Phi^1 = \Phi(\bot)$, $\Phi^2 = \Phi(\Phi(\bot))$, ... describe the loop for at most $0, 1, 2, \ldots$ iterations, and their limit is its meaning. The two styles agree: for every command C and states σ, σ', $\langle C, \sigma\rangle \Downarrow \sigma'$ if and only if $[\![C]\!]\sigma = \sigma'$.

2.6 Exercises

Exercise 2.2. Give the denotational semantics of $E ::= E + E \mid (0..9)^+$ and compute $m[\![12 + 30]\!]$.

Solution. The equations are the ones of this section. Then $m[\![12]\!] = 1 * 10 + 2 = 12$, $m[\![30]\!] = 30$ and $m[\![12 + 30]\!] = 12 +_{\mathbb{N}} 30 = 42$.

Exercise 2.3. Give the semantics of $S ::= (+\ L) \mid (*\ L) \mid \mathit{num}$ and compute $m[\![(+\ 1\ (*\ 2\ 3)\ 4)]\!]$.

Solution. $m[\![L]\!]$ for $1\ (*\ 2\ 3)\ 4$ is the list $(1, 6, 4)$, because $m[\![(*\ 2\ 3)]\!] = \mathit{apply}(*, (2, 3)) = 6$. Then $\mathit{apply}(+, (1, 6, 4)) = 11$.

Exercise 2.4. Give the meaning of $\{1, 3\} \cup \{3, 4\}$ and of $\{\}^c$ as 10-tuples.

Solution. $\{1, 3\} \cup \{3, 4\}$ has *true* at positions 1, 3 and 4, and *false* elsewhere. $\{\}^c$ is the tuple of ten *true*.

Exercise 2.5. Translate `begin x := 3; y := x + 4; x := x * y end` to stack code.

Solution.
```
PUSH 3, ASG x, GET x, PUSH 4, PLUS, ASG y, GET x, GET y, TIMES, ASG x
```

Running it leaves $x = 21$ and $y = 7$, the same memory as the semantics of assignment sequences.

Exercise 2.6. Compute $[\![x := x + 1; y := x * 2]\!]\sigma$ for $\sigma(x) = 3$.

Solution. The first assignment gives $\sigma_1 = \sigma[x \mapsto 4]$, and the second $\sigma_1[y \mapsto 8]$.

3 Axiomatic semantics

This section goes beyond the slides, which only name axiomatic semantics as the third flavor; the pre- and postconditions of dynamic analysis, in the semantic analysis lecture, are its starting point. Axiomatic semantics gives meaning to programs through logical assertions about what is true before and after they run.

3.1 Hoare triples

Definition 3.1. A **Hoare triple** $\{P\}\ C\ \{Q\}$ says that if the precondition P holds before C runs and C terminates, then the postcondition Q holds after it.

For example $\{x = 5\}\ x := x + 1\ \{x = 6\}$, and the triple of the semantic analysis lecture, $\{x \leq 10\}$ `while` $x < 10$ `do` $x := x + 1\ \{x = 10\}$.

3.2 Rules

$$\frac{}{\{P[x := e]\}\ x := e\ \{P\}}\ \textsc{Assign} \qquad \frac{\{P\}\ C_1\ \{R\} \qquad \{R\}\ C_2\ \{Q\}}{\{P\}\ C_1; C_2\ \{Q\}}\ \textsc{Seq}$$

$$\frac{\{P \wedge b\}\ C_1\ \{Q\} \qquad \{P \wedge \neg b\}\ C_2\ \{Q\}}{\{P\}\ \texttt{if}\ b\ \texttt{then}\ C_1\ \texttt{else}\ C_2\ \{Q\}}\ \textsc{If} \qquad \frac{\{I \wedge b\}\ C\ \{I\}}{\{I\}\ \texttt{while}\ b\ \texttt{do}\ C\ \{I \wedge \neg b\}}\ \textsc{While}$$

$$\frac{P \Rightarrow P' \qquad \{P'\}\ C\ \{Q'\} \qquad Q' \Rightarrow Q}{\{P\}\ C\ \{Q\}}\ \textsc{Cons}$$

The assignment rule works backward: to have P after $x := e$, P with e in place of x must hold before. In While, I is the **loop invariant**: it holds before the loop, every iteration preserves it, and when the loop ends $I \wedge \neg b$ must give the postcondition. Cons strengthens preconditions and weakens postconditions to connect rules.

Partial and total correctness. $\{P\}\ C\ \{Q\}$ is partial correctness: it says nothing about termination. Total correctness, written $[P]\ C\ [Q]$, also requires C to terminate, usually proved with a **variant**, a quantity bounded below that decreases at each iteration.

Weakest preconditions. Instead of guessing a precondition, the weakest one can be computed:

$$wp(x := e, Q) = Q[x := e] \qquad wp(C_1; C_2, Q) = wp(C_1, wp(C_2, Q))$$
$$wp(\texttt{if}\ b\ \texttt{then}\ C_1\ \texttt{else}\ C_2, Q) = (b \Rightarrow wp(C_1, Q)) \wedge (\neg b \Rightarrow wp(C_2, Q))$$

For loops there is no mechanical formula; an invariant is needed.

3.3 Example: the factorial

The factorial of Section 1, with *fac* starting at 1 and l at n, has the invariant

$$I : fac \cdot l! = n! \wedge l \geq 0.$$

Before the loop, $1 \cdot n! = n!$. An iteration with $l > 0$ multiplies *fac* by l and decrements l, and $(fac \cdot l) \cdot (l-1)! = fac \cdot l!$, so I still holds. When the loop ends, $l = 0$ and I gives $fac = n!$. The variant is l. Figure 7 places the assertions on the loop.

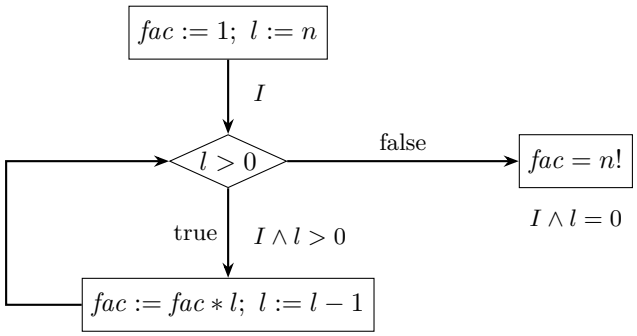


Figure 7: The invariant I on the factorial loop.

For a loop that adds $a[0..n-1]$ into s, the invariant is $s = \sum_{k=0}^{i-1} a[k] \wedge 0 \leq i \leq n$; at the end $i = n$ and s is the full sum.

3.4 Exercises

Exercise 3.2. Prove $\{x = n\}\ y := x * x\ \{y = n^2\}$.

Solution. By Assign, $\{x * x = n^2\}\ y := x * x\ \{y = n^2\}$. Since $x = n \Rightarrow x * x = n^2$, Cons gives the triple.

Exercise 3.3. Compute $wp(x := x + 1, x > 0)$.

Solution. $(x > 0)[x := x + 1] = x + 1 > 0$, that is, $x \geq 0$ on integers.

Exercise 3.4. Is $\{x > 0\}$ `while` $x \neq 0$ `do` $x := x - 1\ \{x = 0\}$ partial or total correctness? Which variant proves termination?

Solution. With invariant $x \geq 0$ it holds as partial correctness: at the end $x \geq 0 \wedge x = 0$. It also terminates: x is a variant, bounded below by 0 and decreasing by one in each iteration.

4 Control flow

The order in which a program executes its parts is essential for most programs. This section follows Scott and Aldrich [7, Ch. 6], Tennent [8, Ch. 6] and Pratt and Zelkowitz [6, Ch. 6]. The lecture lists the mechanisms that order execution (Table 4).

Mechanism	Meaning
sequencing	statements execute in a given order, usually the order in which they are written
selection	a choice between statements (`if`, `case`)
iteration	a fragment of code runs repeatedly (`while`, `for`)
procedural abstraction	a collection of control constructs encapsulated and treated as a single entity
recursion	an expression defined in terms of simpler versions of itself
concurrency	fragments that run at the same time, in parallel or interleaved
nondeterminism	the order of choice among statements is deliberately unspecified; any order is correct

Table 4: Mechanisms of control flow.

4.1 Expression evaluation

Expressions are built from literals, operators and function calls, and the notation varies: infix `a+b`, a call `my_func(A,B,C)`, a method `a.operator+(b)`, prefix `(* (+ 1 3) 2)` in Lisp, keywords in Smalltalk (`myBox displayOn: myScreen at: 100@50`), and an `if` that returns a value (`a := if b <> 0 then a/b else 0`).

Precedence and associativity. Precedence rules say which operators group more tightly in the absence of parentheses; associativity rules say whether operators of equal precedence group to the left or to the right. The expression `a + b * c ** d ** e / f` can be read as

$$((((a+b)*c)**d)**e)/f, \qquad a+((b*(c**(d**e)))/f), \qquad a+(((b*c)**d)**(e/f)).$$

With the usual rules (`**` binds tightest and associates to the right, `*` and `/` bind tighter than `+` and associate to the left), the second reading is the right one (Figure 8). Haskell declares precedence (0 to 9) and associativity for each operator: `infixr 8 ^, **`; `infixl 7 *, /`; `infixl 6 +, -`; `infix 4 ==, <` (non-associative); `infixr 3 &&`; `infixr 2 ||`; `infixr 0 $`.

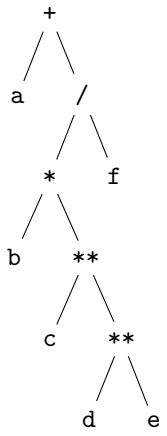


Figure 8: The tree of `a + b * c ** d ** e / f`.

4.2 Assignment

Execution is split into expressions, which return a value and may have side effects (functions), and statements, which are executed for their side effects and return no value (assignment). In `d = a; a = b + c;` the name on the left denotes a location and the expression on the right a value.

Definition 4.1. An **l-value** is an expression that denotes a location (it can appear on the left of an assignment); an **r-value** is an expression that denotes a value.

Value and reference models. In a value model a variable is a named container for a value; in a reference model a variable is a name that refers to a value stored elsewhere (Figure 9). After `b = 2; c = b; a = b + c;`, in the value model each variable holds its own number. In the reference model `b` and `c` refer to the same object 2. Equality has two meanings in the reference model: do two variables refer to the same object (`===` in JavaScript, `==` on objects in Java), or to equal values (`==` in JavaScript, `equals` in Java)?

```
Integer a = 1000, b = 1000;
boolean sameObject = (a == b);
boolean sameValue = a.equals(b);
```

`a == b` is false because `a` and `b` are two objects with the value 1000.

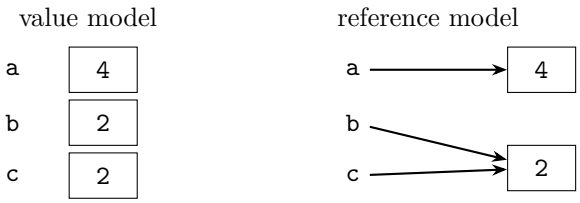


Figure 9: The variables after `b = 2; c = b; a = b + c` in the two models.

Initialization. Assignment (`=`, `:=`) and initialization (aggregates, constructors) are usually different operations. To use a variable in an expression, every control path to the expression must assign it. Java checks this statically. In

```
int j = 3;
int i;
if (j > 0) { i = 2; }
System.out.println(i);
```

`javac` reports `variable i might not have been initialized`: if the condition were false, `i` would have no value. The slide writes the example with `final static int j = 3`. With that declaration `j > 0` is a constant expression whose value is true, Java's rules treat the `if` as always taken, and the program compiles.

Boxing. Many languages mix values and references. Converting a value into an object that wraps it (boxing) and back (unboxing) is needed to store values where references are expected, as in a hash table. Java does it explicitly or automatically:

```
Hashtable<Integer, Integer> ht = new Hashtable<>();
Integer n = Integer.valueOf(13);
ht.put(n, Integer.valueOf(31));
int m = ht.get(n).intValue();
ht.put(14, 32);
int k = ht.get(14);
```

4.3 Evaluation order

In `a - f(b) - c * d`, associativity says that `f(b)` is subtracted from `a` first, and precedence that `c * d` is computed before the second subtraction. Neither says whether `a - f(b)` is evaluated before `c * d`, nor in which order the arguments of `f(a, g(b), c)` are evaluated. The order matters for two reasons. Side effects: if `f(b)` modifies `d`, `c * d` gives different results before and after. Efficiency: in `a := B[i]; c := a*2 + f(b)*3`, computing `a*2` first keeps one register busy during the call, but loading `a` may be slow. Many languages leave the order unspecified so that compilers can choose. The `foo(i++, -i)` of Section 1 is the same problem.

Short-circuit evaluation. In `(a < b) and (b < c)`, if `a < b` is false there is no point in evaluating the rest. Short-circuit evaluation stops as soon as the value is known, and programs rely on it:

```
struct node { int key; struct node *next; };
```

```
struct node *find(struct node *p, int val) {
    while (p && p->key != val)
        p = p->next;
    return p;
}
```

`p->key` is not evaluated when `p` is `NULL`. In small step, short-circuit `||` is two rules that look only at the left operand:

$$\frac{}{\langle \mathit{true} \mid\mid B, s\rangle \to \langle \mathit{true}, s\rangle}\ \textsc{Or-True} \qquad \frac{}{\langle \mathit{false} \mid\mid B, s\rangle \to \langle B, s\rangle}\ \textsc{Or-False}$$

4.4 Structured control flow

Early languages built control flow from `goto` and labels. The lecture's Fortran program for primes jumps with arithmetic `IF`s (three labels for negative, zero and positive) and `GO TO`. Any imperative algorithm can be written with sequence, selection (`if`) and iteration (`for` and `while`), which is the basis of structured programming. Dijkstra's letter "Go To Statement Considered Harmful" [2] argued that unrestricted jumps make it impossible to reason locally about a program.

break and continue. Leaving a loop early, or skipping to the next iteration, are restricted jumps. Pascal uses `goto 100` to a label after the loop; Ada has `exit when` and named loops (`exit Main_Cycle when Found`); Java has labeled `continue`:

```
List<String> out = new ArrayList<>();
outerLoop:
for (int i = 1; i <= 3; i++) {
    for (int j = 1; j <= 3; j++) {
        if (i == 2 && j == 2) {
            continue outerLoop;
        }
        out.add(i + "," + j);
    }
}
```

When $i = 2$ and $j = 2$ the inner loop stops and the outer one continues with $i = 3$, so the pair $(2, 3)$ is never printed.

Early termination. `return` leaves a function before its end, avoiding a `goto` or a control variable that the rest of the function would check.

Multilevel returns. Returning from several nested calls at once requires unwinding the run-time stack. The lecture's Pascal example jumps from `search_file` to a label in the enclosing `search`. Ruby uses `catch :found` around the calls and `throw :found, line` inside them. In Scheme the same behavior is a continuation:

```
(define (search key files)
  (call-with-current-continuation
    (lambda (found)
```

```
      (for-each (lambda (file)
                  (for-each (lambda (line)
                              (if (equal? line key) (found line)))
                            file))
                files)
      "not found")))
```

`found` is the continuation of the whole search: calling it returns from `search` immediately, from inside both loops.

Continuations. A continuation captures the context (the rest of the computation) in which execution may continue. `call/cc` passes the current continuation to a function:

```
(define (f return)
  (return 2)
  3)

(define a (call-with-current-continuation f))
(define b (f (lambda (x) x)))
```

With `call/cc`, `return` is the continuation, so `(return 2)` abandons the rest of `f` and the result is 2. With the identity function as `return`, the call returns normally and `f` gives 3. Continuations are also used to implement web applications where the user goes back and forward between pages (Smalltalk's Seaside).

Exceptions. This paragraph goes beyond the slides. An exception transfers control to the nearest handler:

```
def risky(x):
    if x == 0:
        raise ValueError("boom")
    return 10 // x

try:
    r = risky(0)
    r = 100
except ValueError:
    r = -1
```

In big step, a command can now end in a store or raise a value; a sequence stops at the first raise, and `try` catches it. Exceptions, `break` and `return` are tamed jumps: their possible targets are known statically.

4.5 Sequencing

A sequence (`;`) controls the order of side effects: statements run in the order in which they are written. Statements inside delimiters (`{ ... }`, `begin ... end`) form a compound statement, which with its declarations is a block. A block may create a new environment where its variables exist, as the symbol tables of the previous block. A `static` local variable in C is the opposite case: it is created once and keeps its value between calls, so the same sequence of calls produces a different value each time:

```
void gen_new_name(char *s, char l) {
    static short int name_nums[52];
    int index = (l >= 'a' && l <= 'z') ? l - 'a' : 26 + l - 'A';
    name_nums[index]++;
    sprintf(s, "%c%d", l, name_nums[index]);
}
```

Three calls with `'p'`, `'p'`, `'l'` produce `p1`, `p2` and `l1`.

4.6 Selection

if and else-if. Selection follows one of several paths from a point of the program: `if ... then ... else ...`, with `else if` (or `elif`) for more cases. Lisp's `cond` expresses multiple selection without nesting, like a `switch` on conditions. The lecture's example, in Common Lisp:

```
(defun quadrant (ang)
  (cond ((< ang 90)  "cond1")
        ((< ang 180) "cond2")
        ((< ang 270) "cond3")
        ((< ang 360) "cond4")))
```

The conditions are tested in order and the first true one wins: 354 gives `"cond4"`.

Pattern matching. Conditions can be evaluated indirectly by matching values against patterns. The lecture's Elixir `merge` has one case per shape of the two lists; in Rascal:

```
list[int] merge([], list[int] ys) = ys;
list[int] merge(list[int] xs, []) = xs;
list[int] merge(first:[int x, *xs], second:[int y, *ys])
  = x < y ? [x] + merge(xs, second) : [y] + merge(first, ys);
```

Jump code. When a condition only decides a branch, its value does not have to be computed: the code can jump as soon as the result is known, as in short-circuit evaluation. For `if ((A > B) and (C > D)) or (E ≠ F)`, Figure 10 shows the jumps: if `A > B` is false the `and` is false and the code goes to test `E ≠ F`; if both comparisons are true it jumps directly to the `then` block.

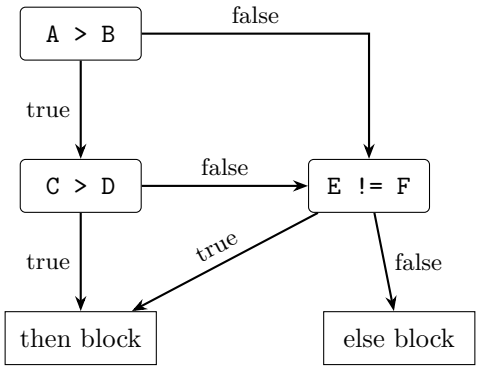


Figure 10: Jump code for `if ((A > B) and (C > D)) or (E ≠ F)`.

4.7 Iteration

Iteration repeats a set of statements, either a given number of times (enumeration) or while a predicate holds.

Enumeration-controlled loops. A loop such as `loop i in range(1, 10) step 2` has an iteration variable, a first and a last value, and a step. The lecture asks four questions about them:

1. Can `i`, `first` or `last` be modified inside the loop? Most languages forbid changing the iteration variable, and evaluate the bounds only once, at the beginning. They protect against "threatening" statements: assigning the loop variable, passing it by reference to a subroutine, or reading it from a file.
2. What happens if `first` is larger than `last`? The loop must not execute: the terminating condition is tested before the first iteration (`if r1 > r2 then exit_loop`). This works for a positive step; a negative one needs an explicit form (`downTo`) so that the compiler generates the comparison the other way around.
3. What is the value of `i` after the loop? After `loop i := 'a' to 'z' do ... end`, `print(i)` could print something undefined, the value after the range (which for a character loop may not even exist), or the last value of the range. Languages answer differently, and many make the variable local to the loop to avoid the question.
4. Can control jump into the loop from outside? Structured languages forbid it.

Figure 11 shows the structure: the bounds are evaluated once, the test comes first, and the step follows the body.

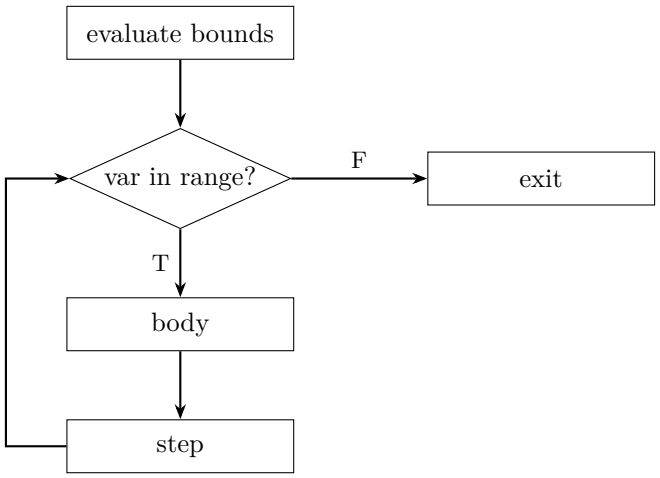


Figure 11: An enumeration-controlled loop.

Logically controlled loops. `loop predicate do loop_body` has no explicit iteration variable and no required increment. The question is where the terminating condition is tested: normally at the beginning (`while`), but it can be at the end (`do-while`, which runs the body at least once) or in the middle (`loop ... exit when ... end`).

Generators. This paragraph goes beyond the slides. A generator produces the values of a loop one at a time and suspends between them:

```
def countdown(n):
    while n > 0:
        yield n
        n -= 1
```

4.8 Recursion

Recursive and iterative programs are equivalent, up to the use of the stack. A call in tail position (the last thing the function does) needs no stack frame of its own, and the compiler can turn it into a jump:

```
int gcd(int a, int b) {
    if (a == b) return a;
    else if (a > b) return gcd(a - b, b);
    else return gcd(a, b - a);
}

int gcd_loop(int a, int b) {
start:
    if (a == b) return a;
    else if (a > b) { a = a - b; goto start; }
    else { b = b - a; goto start; }
}
```

Scheme requires implementations to do this, so a tail-recursive function runs in constant stack space:

```
(define (gcd a b)
  (cond ((= a b) a)
        ((> a b) (gcd (- a b) b))
        (else (gcd a (- b a)))))
```

A function that is not tail-recursive can be transformed into continuation-passing style: each call receives, as an extra argument, a function with the rest of the computation.

```
(define (fact n)
  (if (= n 0) 1 (* n (fact (- n 1)))))

(define (fact-cps n k)
  (if (= n 0)
      (k 1)
      (fact-cps (- n 1) (lambda (v) (k (* n v))))))
```

In `fact-cps`, the multiplication that `fact` does after the recursive call moves into the continuation, and every call is a tail call.

4.9 Nondeterminism

Definition 4.2. A **nondeterministic** abstraction is one in which the choice between alternatives (between control paths) is deliberately unspecified, so that different executions may make different choices.

Concurrency usually introduces nondeterminism: the order in which statements of different threads run is not guaranteed. Dijkstra's guarded commands are the classic sequential example: among several guards that are true, any branch may be taken.

4.10 Exercises

Exercise 4.3. Are these two programs the same?

```
j = index_fn(i);
A[j] = A[j] + 1;
```

```
A[index_fn(i)] = A[index_fn(i)] + 1;
```

Solution. Not necessarily. The second calls `index_fn` twice. If the function has a side effect, the two calls can return different indices, and the program increments one cell with the value of another; the order of the two calls is also unspecified.

```
int A[4] = {0, 0, 0, 0}, B[4] = {0, 0, 0, 0};
calls = 0;
int j = index_fn(1);
A[j] = A[j] + 1;
calls = 0;
B[index_fn(1)] = B[index_fn(1)] + 1;
```

The test uses an `index_fn` that counts its calls.

Exercise 4.4. For `a = b + c; d = c + e + b`, in which order can the expressions be evaluated?

Solution. Addition is commutative and associative on integers, so a compiler may rewrite the second statement as `d = b + c + e` and reuse `b + c`: compute `b + c`, assign it to `a`, compute `a + e`, assign it to `d`. With floating point or side effects the rewrite is not safe.

Exercise 4.5. Draw the control flow diagram of a chain of `else if` that assigns the grades A to F according to the score (at least 90, 80, 70, 60).

Solution. A chain of tests: ≥ 90 goes to A, otherwise ≥ 80 goes to B, and so on; the last `else` goes to F. All branches join before the final statement. In JavaScript:

```
function grade(studentScore) {
  let studentGrade;
  if (studentScore >= 90) {
    studentGrade = "A";
  } else if (studentScore >= 80) {
    studentGrade = "B";
  } else if (studentScore >= 70) {
    studentGrade = "C";
  } else if (studentScore >= 60) {
    studentGrade = "D";
  } else {
    studentGrade = "F";
```

```
  }
  return studentGrade;
}
```

With the score 78 the first two tests fail and the third assigns C.

Exercise 4.6. What does the Java `outerLoop` example print? And with `break outerLoop` instead of `continue outerLoop`?

Solution. With `continue`: $(1,1)$, $(1,2)$, $(1,3)$, $(2,1)$, $(3,1)$, $(3,2)$, $(3,3)$. With `break` both loops end at $(2,2)$, so it prints $(1,1)$, $(1,2)$, $(1,3)$, $(2,1)$.

5 Environments, scoping and parameter passing

This section follows Scott and Aldrich [7, Ch. 3], Friedman and Wand [4, Sec. 3.5–3.7], Lee [5, Sec. 5.9] and Aho et al. [1, Sec. 1.6].

5.1 Variables and bindings

Variables appear in a program as declarations or as references. A **declaration** introduces a variable into the program and makes it available at run time; a **reference** is a use of the variable. The lecture's declarations come from many languages: `val imAVar = 3`, `int myVar;`, `(defvar x 42)`, `let var = "string";`, `variable_name = 2.71`, `var aVar string = "variable"`, `AVar2 := "new declaration"` and the parameter of `(lambda (x) x)`. A variable is bound by the declaration that declares it, and through it to its value.

5.2 Scopes

Definition 5.1. The **scope** of a declaration is the part of the program where the variable is valid and visible.

Scopes manage declarations (as symbol tables do) and let the same name be reused without undesired side effects. In

```
(define res
  (let ((x 3)
        (y 4))
    (+ (let ((x (+ y 5)))
         (* x y))
       x)))
```

the outer `let` creates a scope with $x = 3$ and $y = 4$, the inner one a scope with $x = y + 5 = 9$, and each application of `*` and `+` runs in a scope of its own. Figure 12 shows the environments when `(* x y)` runs: `x` is found in the inner environment and `y` in the outer one. The product is 36, and the outer sum adds the outer `x`: the result is 39.

Each scope creates an environment for the execution of the program, with its rules of validity and visibility. The lecture compares three rules. With static scoping, a name is bound by the closest declaration in the text, moving outward. With dynamic scoping, it is bound according to where the

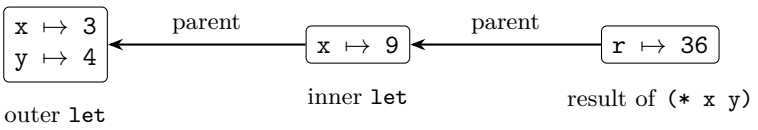


Figure 12: Environments while evaluating `(* x y)`.

function is called, so different calls may give different results. Qualified names group names under a prefix to avoid collisions between scopes.

5.3 Static scoping

Definition 5.2. The **static (lexical) scoping** rule: to find the declaration of a use of a variable, search outward from the scope of the use, through the enclosing scopes, until a declaration is found. Variables that follow this rule are **lexical variables**.

Scopes are delimited by code blocks (`{...}`): blocks, function definitions, control statements and objects each create a scope nested in the scope where they are declared, with its own memory space. Function definitions create a scope in which each parameter is a variable, plus the variables of the body. A function runs in the scope where it was defined: its free variables are bound by the declarations around its definition, not around its call.

```
(define x 10)
(define (f) x)
(define (g)
  (let ((x 20))
    (f)))
```

`(g)` returns 10: the `x` in the body of `f` is the global one, the declaration around the definition of `f`, even though `f` is called from inside the `let` of `g`.

Holes and shadows. A declaration in an inner scope with the same name as an outer one **shadows** it: inside the inner scope the outer variable is not visible, and the region where it is hidden is a hole in its scope.

```
int a = 1;
int b = 1;
{
    int b = 2;
    {
        int a = 3;
        cout << a << b << " ";
    }
    {
        int b = 4;
        cout << a << b << " ";
    }
    cout << a << b << " ";
}
cout << a << b;
```

The program prints `3 2`, `1 4`, `1 2` and `1 1` (Figure 13). In the first inner block, `a = 3` shadows the outer `a` and `b = 2` is visible from the enclosing block. In the second, `b = 4` shadows `b = 2` and the outer `a` is visible. After the blocks, each name refers to its outer declaration again.

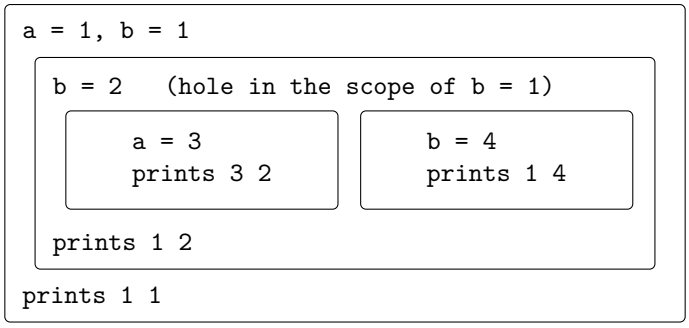


Figure 13: Nested blocks of the shadowing example and what each `cout` prints.

Extent. The **extent** of a binding is the time during which it is valid. Bindings are usually valid while the block that defines them runs, but they can live longer, even indefinitely, when a closure that refers to them is returned:

```
(define (make-counter)
  (let ((count 0))
    (lambda ()
      (set! count (+ count 1))
      count)))
```

`count` belongs to a call of `make-counter` that has finished, but the returned function still uses it. Scope is a property of the text; extent is a property of the execution.

Lexical depth. As with symbol tables, each new block has its own environment with a link to its parent. In `let x = exp1 in let y = exp2 in -(x, y)`, the environment of the body has y at depth 0 and x at depth 1 (Figure 14). Because scoping is static, the depth of each use is known before the program runs, and a compiler can replace each name with a pair (depth, position): a lexical address.

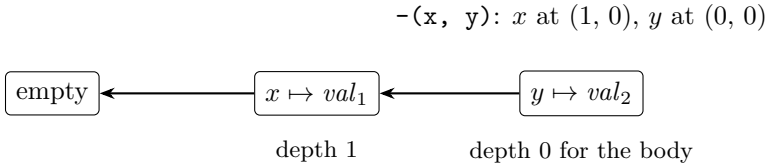


Figure 14: Environments and lexical depth for `let x = exp1 in let y = exp2 in -(x, y)`.

Summing up the rule: variables get their meaning from their location in the code. Look for the variable in the current environment; if it is found, use it; if not, go to the next enclosing scope and repeat. Operations with lexical scope always do the same thing, wherever they are called.

5.4 Dynamic scoping

Definition 5.3. The **dynamic scoping** rule: the value of a variable is the one of the most recently called procedure that is still active and declares the variable.

The lecture shows the difference in Perl:

```
our $x = 10;
sub f { return $x; }
sub g { local $x = 20; return f(); }
```

With static scoping, the `$x` of `f` is the global one and `g()` returns 10. With dynamic scoping, when `f` runs the most recent active function that declares `$x` is `g`, so `g()` returns 20; Perl's `local` gives exactly this behavior, and the test checks that `g()` is 20 and `f()` alone is 10. In Common Lisp, variables declared with `defvar` are dynamically scoped (special), and the same program reads:

```
(defvar *x* 10)

(defun f () *x*)

(defun g ()
  (let ((*x* 20))
    (f)))
```

A variable bound with `let` that was not declared special is lexical:

```
(let ((y 10))
  (defun f-lex () y))

(defun g-lex ()
  (let ((y 20))
    (declare (ignorable y))
    (f-lex)))
```

`(g-lex)` returns 10. Shell variables are also dynamic:

```
x=10
f() { echo "$x"; }
g() { local x=20; f; }
```

The lecture's second Perl example adds `sub h(param) { local $x = param + 5; return f(); }` and calls `h(rand(100))` from `g`: now `f` sees the `$x` of `h`, which is different in every call. The behavior of an operation with dynamic scope can change with every call.

Logo. The third example is in Logo, which uses dynamic scoping. A procedure `test1` with parameter `:x` only calls `test2`; `test2` prints `In test2:` followed by `:x`. The program sets the global `x` to `"foo` and calls `test1 "bar`. `test2` called alone prints `foo`, the global value. Called from `test1 "bar`, it prints `bar`: the most recent active procedure with `x` is `test1`, whose parameter `x` is `"bar`.

```
(defvar *lx* "foo")
(defun test2 () *lx*)
(defun test1 (*lx*) (test2))
```

Advantages and disadvantages. Dynamic scoping favors flexibility: variables can be accessed from anywhere, which can help reuse and adaptation of code. It is harder to reason about, the flow is harder to understand, it can be slower (no static optimizations such as lexical addresses), and it is error-prone because of unintended accesses to variables.

5.5 Qualified names and access modifiers

Global variables can be accessed from the whole program, which causes name collisions. With **qualified names**, related names are grouped under a name, the prefix, and a variable has two sets of scopes: a scope (usually the global one) where the qualified name `Group.name` is visible, and one or more narrower scopes where the unqualified name is visible as well. Modules, packages and namespaces work this way.

Access modifiers restrict visibility further, beyond the scoping rules. A private member is only visible inside the object (it is part of the implementation); a public member is visible anywhere (it is part of the interface). The details depend on the language; Table 5 shows Swift's levels.

Modifier	Immediate scope	File	Containing module	Rest of the world
`open`, `public`	yes	yes	yes	yes
`internal`	yes	yes	yes	no
`fileprivate`	yes	yes	no	no
`private`	yes	no	no	no

Table 5: Where a declaration is visible for each access modifier in Swift. `open` also allows subclassing outside the module.

5.6 Environments and stores

This subsection goes beyond the slides. An **environment** $\rho : \mathit{Var} \to \mathit{Loc}$ maps names to locations, and a **store** $\sigma : \mathit{Loc} \to \mathit{Val}$ maps locations to values; reading x is $\sigma(\rho(x))$. SIMP used a single map, $\mathit{Var} \to \mathit{Val}$, but separating the two is what allows two names to refer to the same location (a reference parameter, a closure) and one name to refer to different locations at different times (a recursive call, a shadowing declaration). The parameter passing modes below are differences in how ρ and σ are extended.

Closures in loops. A closure captures a binding, not a value. In Python a loop has one variable for all its iterations; JavaScript's `let` creates a new one in each iteration:

```
funcs = []
for i in range(3):
    funcs.append(lambda: i)

fixed = []
for i in range(3):
    fixed.append(lambda i=i: i)
```

```
var funcsVar = [];
for (var i = 0; i < 3; i++) {
  funcsVar.push(() => i);
```

```
}
var funcsLet = [];
for (let j = 0; j < 3; j++) {
  funcsLet.push(() => j);
}
```

5.7 Parameter passing

The parameters written in a call are the **actual** parameters; the parameters in the definition of the function are the **formal** parameters. The modes differ in what the formal parameter is bound to.

Call by value. The actual parameters are evaluated, and their values copied into the space of the formal parameters (the space of the function). Changes to the formal parameters do not affect the caller.

Call by reference. The address of the actual parameter is passed, and the function uses it directly. The formal parameter is another name for the caller's variable.

```
void swapByValue(int x, int y) { int t = x; x = y; y = t; }
void swapByReference(int &x, int &y) { int t = x; x = y; y = t; }
```

In the lecture's program, `swap(x, y)` with $x = 1$, $y = 2$ prints `After Swapping in function x: 2, y: 1` in both cases, but `main` prints `x: 1, y: 2` with call by value and `x: 2, y: 1` with call by reference.

Call by name. Call by name is between values and references: the function sees the parameter as if it were a copy, but the argument expression is never evaluated at the call; it is evaluated, in the caller's environment, each time the parameter is used (with variables renamed if needed). With `int x = 1, y = 2+8;` and `swap(x, y)` by name, `t = x` reads 1, `x = y` evaluates the expression of `y` and writes 10, `y = t` writes 1: the program prints `x: 10, y: 1`. In Scheme the names can be passed as functions that read and write the caller's variables:

```
(define (swap-by-name get-x set-x! get-y set-y!)
  (let ((t (get-x)))
    (set-x! (get-y))
    (set-y! t)))

(define (main-by-name)
  (let ((x 1) (y (+ 2 8)))
    (swap-by-name (lambda () x) (lambda (v) (set! x v))
                  (lambda () y) (lambda (v) (set! y v)))
    (list x y)))
```

Passing parameters between procedures. The lecture compares the modes with one program:

```
Program Test
  Var x: int, y: int; v[5]: int;
  Proc p1(val a: int, ref b: int, ref c: int)
```

```
  Begin
    b = b + 1;
    c = a;
  end
Begin
  v = [0,1,2,3,4]
  x = 10;
  y = 1
  p1(x, y, v[y]);
end
```

and then changes the mode of `b` and `c`. Table 6 gives the result of each version.

Modes of a, b, c	$b = b + 1$	$c = a$	At the end
val, ref, ref	$y = 2$	$v[1] = 10$ (c was bound to $v[1]$ at the call)	$y = 2$, $v = [0, 10, 2, 3, 4]$
val, ref, name	$y = 2$	$v[2] = 10$ ($v[y]$ is evaluated now, with $y = 2$)	$y = 2$, $v = [0, 1, 10, 3, 4]$
val, VR, name	local $b = 2$	$v[1] = 10$ (y is still 1)	$y = 2$ (copied back), $v = [0, 10, 2, 3, 4]$

Table 6: The program `Test` with three combinations of modes. a is always $x = 10$.

Value-result (copy-restore, VR) copies the value in at the call, works on a local copy and copies it back at the return. With `b` by value-result the change to y is not visible during the call, so `v[y]` still means `v[1]`. In C++ a reference parameter is `int&`, a parameter by name can be a function that returns a reference, and value-result is a copy made by hand:

```
void p1Ref(int a, int &b, int &c) { b = b + 1; c = a; }

void p1Name(int a, int &b, std::function<int&()> c) { b = b + 1; c() = a; }

void p1VR(int a, int &bOut, std::function<int&()> c) {
    int b = bOut;
    b = b + 1;
    c() = a;
    bOut = b;
}
```

Other kinds. The lecture also lists call by move (Emerald), adaptive parameter passing, streaming semantics, cache semantics and lazy semantics or call by need. Call by need is call by name where the argument is evaluated at most once and the value is remembered; the runtime block studies it.

5.8 A small-step rule for calls

This subsection goes beyond the slides. A procedure value is a closure $\mathbf{proc}(x, C, \rho_{def})$: the parameter, the body and the environment where the procedure was defined. Call by value allocates

a new location for the parameter:

$$\frac{\sigma(\rho(f)) = \mathbf{proc}(x, C, \rho_{def}) \qquad \langle E, \rho, \sigma\rangle \Downarrow v \qquad \ell \notin \mathrm{dom}(\sigma) \qquad \langle C, \rho_{def}[x \mapsto \ell], \sigma[\ell \mapsto v]\rangle \Downarrow \sigma''}{\langle \mathbf{call}\ f(E), \rho, \sigma\rangle \to \langle \texttt{skip}, \rho, \sigma''\rangle}\ \textsc{Call-By-Value}$$

The argument is evaluated in the caller's environment ρ, and the body runs in the closure's environment ρ_{def}: that is static scoping. With dynamic scoping the body would run in $\rho[x \mapsto \ell]$, the caller's environment. Call by reference binds x to the existing location of the argument instead of a new one: $\rho_{def}[x \mapsto \rho(y)]$.

5.9 Exercises

Exercise 5.4. Give the values of the variables in each step of `(let ((x 3) (y 4)) (+ (let ((x (+ y 5))) (* x y)) x))`.

Solution. The outer `let` binds $x = 3$, $y = 4$. The inner one evaluates `(+ y 5)` in the outer environment, 9, and binds $x = 9$. The product reads $x = 9$ (inner) and $y = 4$ (outer): 36. Back in the outer scope, the sum reads $x = 3$: $36 + 3 = 39$.

Exercise 5.5. What does the shadowing program print if the declaration `int b = 2;` is replaced by the assignment `b = 2;`?

Solution. Now there is only one `b`, and the assignment changes it. The prints are `3 2`, `1 4`, `1 2` and, at the end, `1 2`: the last line sees the `b` that was assigned 2.

Exercise 5.6. With dynamic scoping, what does `g()` return in the Perl example if `g` calls `h(7)` and `h` calls `f`?

Solution. When `f` runs, the active procedures are `g` (with `$x = 20`) and `h` (with `$x = 7 + 5 = 12`). The most recent is `h`, so the result is 12. With static scoping it would be 10.

Exercise 5.7. Run `test(i, a[i], i)` with $a = [0, 10, 20, 40]$, $i = 0$, and `proc test(ref x, name y, VR z) { x := x + 1; y := y + 1; z := z + 2; y := y + 1 }`.

Solution. x is another name for i, y re-evaluates `a[i]` at each use, and z starts as a local copy of i, 0. `x := x + 1` makes $i = 1$. `y := y + 1` evaluates `a[i]` with $i = 1$: $a[1] = 11$. `z := z + 2` gives $z = 2$, locally. The second `y := y + 1` again uses $a[1]$: 12. At the return, value-result copies z back into i: $i = 2$, which overwrites the change made through x. The final state is $i = 2$, $a = [0, 12, 20, 40]$.

References

[1] Alfred V. Aho, Monica S. Lam, Ravi Sethi, and Jeffrey D. Ullman. *Compilers: Principles, Techniques, and Tools*. Addison-Wesley, 2nd edition, 2007.

[2] Edsger W. Dijkstra. Go to statement considered harmful. *Communications of the ACM*, 11(3):147–148, 1968.

[3] Maribel Fernández. *Programming Languages and Operational Semantics: A Concise Overview*. Springer, 2014.

[4] Daniel P. Friedman and Mitchell Wand. *Essentials of Programming Languages*. The MIT Press, 3rd edition, 2008.

[5] Kent D. Lee. *Foundations of Programming Languages*. Springer, 2nd edition, 2017.

[6] Terrence W. Pratt and Marvin V. Zelkowitz. *Programming Languages: Design and Implementation*. Prentice Hall, 4th edition, 2001.

[7] Michael L. Scott and Jonathan Aldrich. *Programming Language Pragmatics*. Morgan Kaufmann, 5th edition, 2025.

[8] R. D. Tennent. *Principles of Programming Languages*. Prentice-Hall, 1981.